Machine Learning for Biological Sequences

HMM, RNN, LLM: A long sequence of models...

Diego Milone

3ra Escu3la CICADA

tokenpal9za

sinc(*i*)

Instituto de investigación en señales e inteligencia computacional

FICH, Universidad Nacional del Litoral
Consejo Nacional de Investigaciones Científicas y Técnicas, Argentina

02/11/26

Biological sequences: the questions

Biological sequences: the questions

What does it resemble?

—○ search, comparison, clustering, phylogeny, assembly

What is at each position?

—○ positional annotation (genes, exons, domains, sites)

How does it fold?

—○ secondary and 3D structure

What is it, and what does it do?

—○ family, species, localization, function

Can I write a new one?

—○ design and generation

Where does the answer come from?

$\looparrowright$ parameters?

Where does the answer come from?

- String algorithms: the sequence itself, as a discrete object
  - —∘ edit distance (Levenshtein); compressed suffix indexing (BWA, Bowtie); de Bruijn and overlap graphs (SPAdes, Canu)
  - ⇝ almost no parameters

Where does the answer come from?

- String algorithms: the sequence itself, as a discrete object
- Grammars and rules: a formal description of the pattern
  - —○ sequence patterns (PROSITE); stochastic context-free grammars (Infernal, Rfam)
  - ⇝ hand-curated; estimated from families

Where does the answer come from?

- String algorithms: the sequence itself, as a discrete object
- Grammars and rules: a formal description of the pattern
- Homology and evolution: annotated sequences or shared ancestry
  - —○ local alignment (BLAST); profile search (HMMER, MMseqs2); direct coupling analysis (EVcouplings)
  - ⇝ substitution matrices; MSA statistics

Where does the answer come from?

- String algorithms: the sequence itself, as a discrete object
- Grammars and rules: a formal description of the pattern
- Homology and evolution: annotated sequences or shared ancestry
- Biophysics: a model of the physical world

  $\multimap$ nearest-neighbour thermodynamics (RNAfold); energy functions and conformational search (Rosetta); molecular dynamics (GROMACS, AMBER)

  $\looparrowright$ measured in the laboratory

Where does the answer come from?

- String algorithms: the sequence itself, as a discrete object
- Grammars and rules: a formal description of the pattern
- Homology and evolution: annotated sequences or shared ancestry
- Biophysics: a model of the physical world
- Learning: a model from a set of examples

  ⊸ HMM & CRF, 1D-CNN & dilated convolutions, RNN, Transformers, transfer learning & FM...

  ⇝ fitted by optimization, data-driven

The raw data: 3 alphabets, 3 scales

The raw data: 3 alphabets, 3 scales

- DNA: 4 symbols, 10^3 to 10^9 long
  - annotation: gene finding, exons, splice sites, repeats and transposable elements
  - regulation: promoters, enhancers, binding sites, chromatin accessibility, methylation, expression from sequence
  - variation and identity: variant effect, taxonomic assignment, metagenomic binning
  - engineering: CRISPR guide design, synthetic promoters, primer design

The raw data: 3 alphabets, 3 scales

- DNA: 4 symbols, 10^3 to 10^9 long
  - annotation: gene finding, exons, splice sites, repeats and transposable elements
  - regulation: promoters, enhancers, binding sites, chromatin accessibility, methylation, expression from sequence
  - variation and identity: variant effect, taxonomic assignment, metagenomic binning
  - engineering: CRISPR guide design, synthetic promoters, primer design
- RNA: 4 symbols, 10^2 to 10^4 long
  - structure: secondary structure, pseudoknots, 3D structure
  - detection and identity: non-coding RNA detection (miRNA, lncRNA, snoRNA), family assignment
  - regulation and fate: miRNA targets, RNA-protein binding, modifications (m6A), A-to-I editing, stability
  - design: therapeutic mRNA, aptamers, riboswitches

The raw data: 3 alphabets, 3 scales

- DNA: 4 symbols, 10^3 to 10^9 long
  - annotation: gene finding, exons, splice sites, repeats and transposable elements
  - regulation: promoters, enhancers, binding sites, chromatin accessibility, methylation, expression from sequence
  - variation and identity: variant effect, taxonomic assignment, metagenomic binning
  - engineering: CRISPR guide design, synthetic promoters, primer design
- RNA: 4 symbols, 10^2 to 10^4 long
  - structure: secondary structure, pseudoknots, 3D structure
  - detection and identity: non-coding RNA detection (miRNA, lncRNA, snoRNA), family assignment
  - regulation and fate: miRNA targets, RNA-protein binding, modifications (m6A), A-to-I editing, stability
  - design: therapeutic mRNA, aptamers, riboswitches
- Protein: 20 symbols, 10^2 to 10^3 long
  - structure: 3D structure of monomers and complexes, secondary structure, intrinsic disorder
  - function and identity: function (GO terms), enzyme class, family and domain assignment, remote homology
  - localization: subcellular localization, signal peptides, transmembrane topology
  - interaction: binding sites, catalytic residues, interfaces, protein-ligand, antibody-antigen, peptide-MHC
  - modification and variation: post-translational modifications, variant effect, stability, solubility
  - design: de novo design, binder design, directed evolution

The bottleneck is the label, not the sequence

The bottleneck is the label, not the sequence

150 M protein sequences

—○ UniProtKB

The bottleneck is the label, not the sequence

150 M protein sequences

—○ UniProtKB

575 k of them were curated by a human

—○ Swiss-Prot (that is 0.4%, the other 99.6% were never checked by a curator)

The bottleneck is the label, not the sequence

150 M protein sequences

—○ UniProtKB

575 k of them were curated by a human

—○ Swiss-Prot (that is 0.4%, the other 99.6% were never checked by a curator)

256 k have an experimental structure

—○ PDB (that is 0.17%)

The bottleneck is the label, not the sequence

150 M protein sequences

—○ UniProtKB

575 k of them were curated by a human

—○ Swiss-Prot (that is 0.4%, the other 99.6% were never checked by a curator)

256 k have an experimental structure

—○ PDB (that is 0.17%)

10 k RNA structures: this is the hard case

—○ and highly imbalanced across families: the top 10 cover 87% of them (5% in proteins)

The bottleneck is the label, not the sequence

150 M protein sequences

—○ UniProtKB

575 k of them were curated by a human

—○ Swiss-Prot (that is 0.4%, the other 99.6% were never checked by a curator)

256 k have an experimental structure

—○ PDB (that is 0.17%)

10 k RNA structures: this is the hard case

—○ and highly imbalanced across families: the top 10 cover 87% of them (5% in proteins)

Self-supervised pre-training and transfer learning in biological sequences are not just a fad: they are a necessity!

And... compared to natural language?

And... compared to natural language?

- A tiny alphabet: 4 or 20 symbols, against tens of thousands of pieces in a language vocabulary.

And... compared to natural language?

- A tiny alphabet: 4 or 20 symbols, against tens of thousands of pieces in a language vocabulary.
- No word boundaries, no punctuation, no sentences: finding the units is part of the problem.

And... compared to natural language?

- A tiny alphabet: 4 or 20 symbols, against tens of thousands of pieces in a language vocabulary.
- No word boundaries, no punctuation, no sentences: finding the units is part of the problem.
- The syntax is geometry (not just order): symbols interact when they are close in space.

And... compared to natural language?

- A tiny alphabet: 4 or 20 symbols, against tens of thousands of pieces in a language vocabulary.
- No word boundaries, no punctuation, no sentences: finding the units is part of the problem.
- The syntax is geometry (not just order): symbols interact when they are close in space.
- The corpus is a family tree (not an i.i.d. sample): sequences descend from one another.

And... compared to natural language?

- A tiny alphabet: 4 or 20 symbols, against tens of thousands of pieces in a language vocabulary.
- No word boundaries, no punctuation, no sentences: finding the units is part of the problem.
- The syntax is geometry (not just order): symbols interact when they are close in space.
- The corpus is a family tree (not an i.i.d. sample): sequences descend from one another.
- Much of the sequence has no function at all: neutrality is a legitimate state, not noise.

And... compared to natural language?

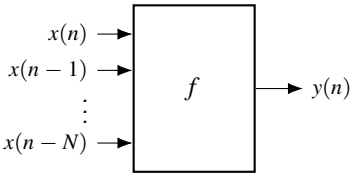
- A tiny alphabet: 4 or 20 symbols, against tens of thousands of pieces in a language vocabulary.
- No word boundaries, no punctuation, no sentences: finding the units is part of the problem.
- The syntax is geometry (not just order): symbols interact when they are close in space.
- The corpus is a family tree (not an i.i.d. sample): sequences descend from one another.
- Much of the sequence has no function at all: neutrality is a legitimate state, not noise.
- Ground truth is expensive: it comes from laboratories (not from native speakers).

And... compared to natural language?

- A tiny alphabet: 4 or 20 symbols, against tens of thousands of pieces in a language vocabulary.
- No word boundaries, no punctuation, no sentences: finding the units is part of the problem.
- The syntax is geometry (not just order): symbols interact when they are close in space.
- The corpus is a family tree (not an i.i.d. sample): sequences descend from one another.
- Much of the sequence has no function at all: neutrality is a legitimate state, not noise.
- Ground truth is expensive: it comes from laboratories (not from native speakers).
- Sequences are read in one direction, but they were not written that way.

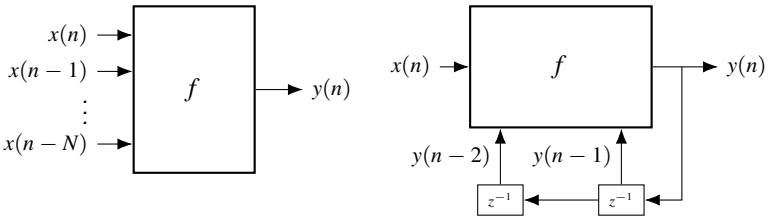
Where does the context come from?

Where does the context come from?



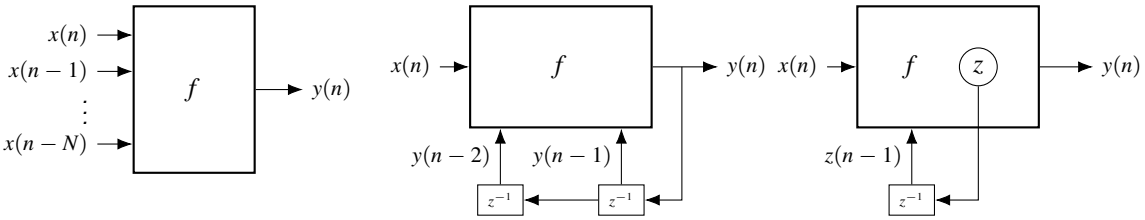
1. Shifted inputs: $y(n) = f(\mathbf{x}(n))$
   - a fixed window over the input (MLP, TDNN, 1D CNN...)

Where does the context come from?



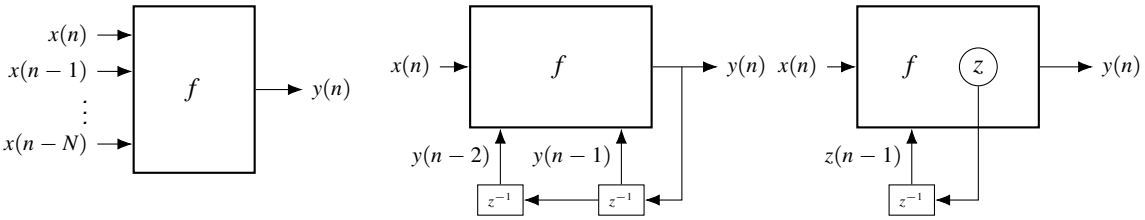
1. Shifted inputs: $y(n) = f(\mathbf{x}(n))$
   - ⊸ a fixed window over the input (MLP, TDNN, 1D CNN...)
2. Output feedback: $y(n) = f(\mathbf{x}(n), \mathbf{y}_1(n))$
   - ⊸ the network sees what it produced (Hopfield, Jordan, AR-LLM...)

Where does the context come from?



1. Shifted inputs: $y(n) = f(\mathbf{x}(n))$
   - a fixed window over the input (MLP, TDNN, 1D CNN...)
2. Output feedback: $y(n) = f(\mathbf{x}(n), \mathbf{y}_1(n))$
   - the network sees what it produced (Hopfield, Jordan, AR-LLM...)
3. Internal states: $y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n))$
   - latent or hidden variables (HMM, CRF, Elman, LSTM, state-space models...)

Where does the context come from?



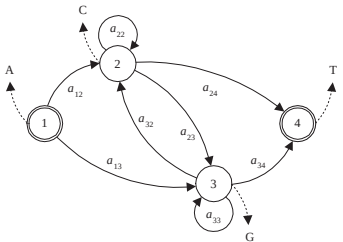
1. Shifted inputs: $y(n) = f(\mathbf{x}(n))$
   - ⊸ a fixed window over the input (MLP, TDNN, 1D CNN... **Transformers!)**
2. Output feedback: $y(n) = f(\mathbf{x}(n), \mathbf{y}_1(n))$
   - ⊸ the network sees what it produced (Hopfield, Jordan, AR-LLM...)
3. Internal states: $y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n))$
   - ⊸ latent or hidden variables (HMM, CRF, Elman, LSTM, state-space models...)

General case: $y(n) = f(\mathbf{x}(n), \mathbf{z}_1(n), \mathbf{y}_1(n))$

Hidden Markov models

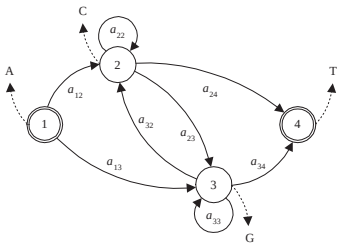
(HMM)

Probabilistic finite-state automata



$$A = \{a_{ij}\} = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 \\ 0 & 1/4 & 1/4 & 1/2 \\ 0 & 1/2 & 1/4 & 1/4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

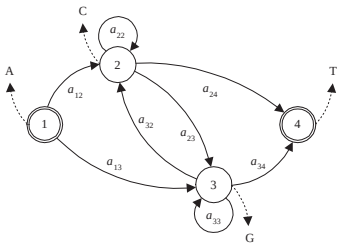
Probabilistic finite-state automata



$$A = \{a_{ij}\} = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 \\ 0 & 1/4 & 1/4 & 1/2 \\ 0 & 1/2 & 1/4 & 1/4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Do all output sequences have the same probability?

Probabilistic finite-state automata



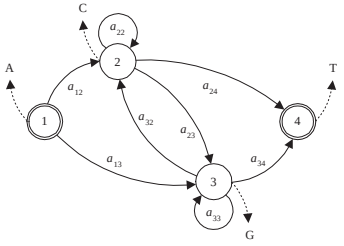
$$A = \{a_{ij}\} = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 \\ 0 & 1/4 & 1/4 & 1/2 \\ 0 & 1/2 & 1/4 & 1/4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Do all output sequences have the same probability?

Given the output sequence A, C, C, G, C, T,

$\Rightarrow$ the transitions are $1 \to 2,\ 2 \to 2,\ 2 \to 3,\ 3 \to 2,\ 2 \to 4$.

Probabilistic finite-state automata



$$A = \{a_{ij}\} = \begin{bmatrix} 0 & 1/2 & 1/2 & 0 \\ 0 & 1/4 & 1/4 & 1/2 \\ 0 & 1/2 & 1/4 & 1/4 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Do all output sequences have the same probability?

Given the output sequence A, C, C, G, C, T,

$\Rightarrow$ the transitions are $1 \to 2,\ 2 \to 2,\ 2 \to 3,\ 3 \to 2,\ 2 \to 4$.

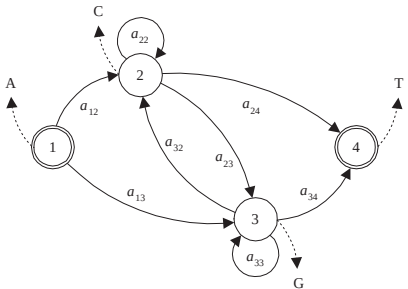
Then, its probability is

$$p_{122324} = a_{12}\ a_{22}\ a_{23}\ a_{32}\ a_{24} = \frac{1}{2} \cdot \frac{1}{4} \cdot \frac{1}{4} \cdot \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{128}$$

Observable vs hidden states

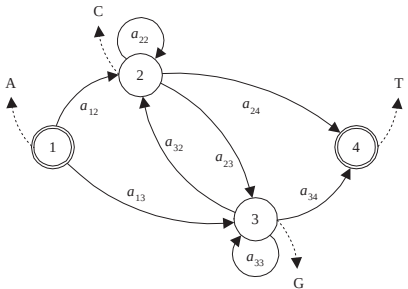
Observable vs hidden states

The observable model



Observable vs hidden states

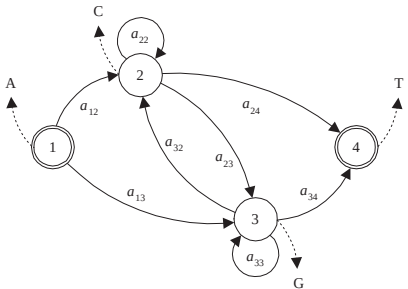
The observable model



In the *observable* model, the state is fully determined at each time step.

Observable vs hidden states

The observable model

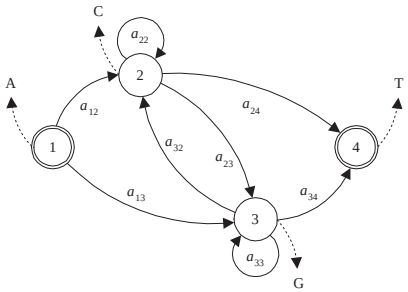


In the *observable* model, the state is fully determined at each time step.

—∘ For example, in this simple model the first symbol is always A and the last one is always T.

Observable vs hidden states

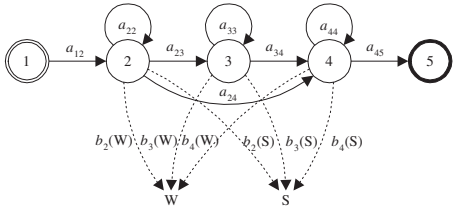
The observable model



In the *observable* model, the state is fully determined at each time step.

—∘ For example, in this simple model the first symbol is always A and the last one is always T.

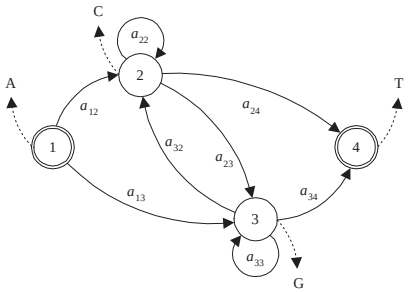
The hidden model



In a *hidden* model, the states must be inferred from the observations.

Observable vs hidden states

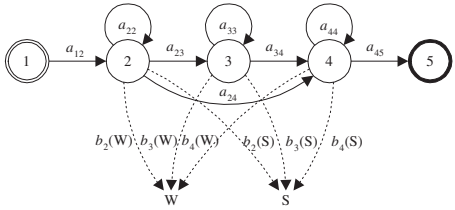
The observable model



In the *observable* model, the state is fully determined at each time step.

—∘ For example, in this simple model the first symbol is always A and the last one is always T.

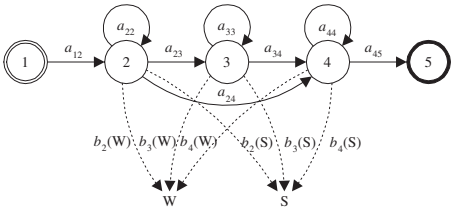
The hidden model



In a *hidden* model, the states must be inferred from the observations.

—∘ Each state can emit any output symbol, and the observation is a probabilistic function of the state.

All the alternative paths: the likelihood



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

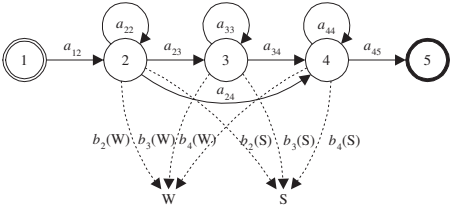
States: regions richer or poorer in GC. The decoded path (of states) is the segmentation of the sequence.

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

W (weak): A or U, two H-bonds

S (strong): G or C, three H-bonds

All the alternative paths: the likelihood



Given the output sequence: W, W, S, W.

—○ What is the probability that this model generated it?

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

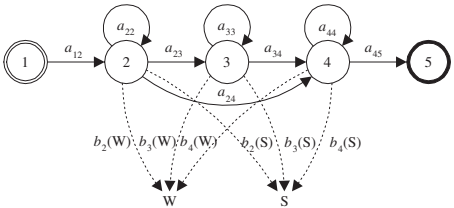
States: regions richer or poorer in GC. The decoded path (of states) is the segmentation of the sequence.

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

W (weak): A or U, two H-bonds

S (strong): G or C, three H-bonds

All the alternative paths: the likelihood



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

States: regions richer or poorer in GC. The decoded path (of states) is the segmentation of the sequence.

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

W (weak): A or U, two H-bonds

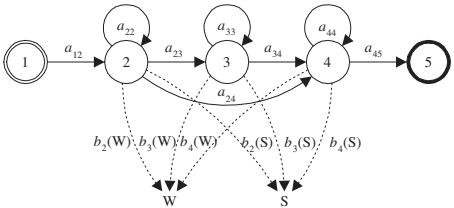
S (strong): G or C, three H-bonds

Given the output sequence: W, W, S, W.

- What is the probability that this model generated it?

But we do not know the state sequence: it is hidden.

All the alternative paths: the likelihood



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

States: regions richer or poorer in GC. The decoded path (of states) is the segmentation of the sequence.

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

W (weak): A or U, two H-bonds

S (strong): G or C, three H-bonds

Given the output sequence: W, W, S, W.

—○ What is the probability that this model generated it?

But we do not know the state sequence: it is hidden.

Two solutions:

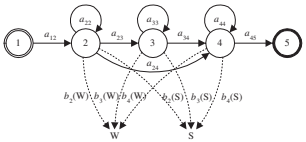
1. Sum over all possible state sequences (law of total probability).
2. Use only the most probable sequence of hidden states.

All the alternative paths: the likelihood

Given the output sequence $\mathbf{x} =$ W, W, S, W.

$\mathbf{q} = 1, q_1, q_2, q_3, q_4, 5$	$\prod_t a_{q_{t-1}q_t}$	$\prod_t b_{q_t}(x_t)$	$\prod_t b_{q_t}(x_t)\, a_{q_{t-1}q_t}$
1, 2, 2, 2, 4, 5	$1 \frac{1}{4}\frac{1}{4}\frac{1}{2}\frac{1}{2} = \frac{1}{64}$	$\frac{1}{3}\frac{1}{3}\frac{2}{3}\frac{2}{3} = \frac{4}{81}$	$\frac{1}{1296}$
1, 2, 2, 3, 4, 5	$1 \frac{1}{4}\frac{1}{4}\frac{1}{2}\frac{1}{2} = \frac{1}{64}$	$\frac{1}{3}\frac{1}{3}\frac{4}{5}\frac{2}{3} = \frac{8}{135}$	$\frac{1}{1080}$
1, 2, 2, 4, 4, 5	$1 \frac{1}{4}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{32}$	$\frac{1}{3}\frac{1}{3}\frac{1}{3}\frac{2}{3} = \frac{2}{81}$	$\frac{1}{1296}$
1, 2, 3, 3, 4, 5	$1 \frac{1}{4}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{32}$	$\frac{1}{3}\frac{1}{5}\frac{4}{5}\frac{2}{3} = \frac{8}{225}$	$\frac{1}{900}$
1, 2, 3, 4, 4, 5	$1 \frac{1}{4}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{32}$	$\frac{1}{3}\frac{1}{5}\frac{1}{3}\frac{2}{3} = \frac{2}{135}$	$\frac{1}{2160}$
1, 2, 4, 4, 4, 5	$1 \frac{1}{2}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{16}$	$\frac{1}{3}\frac{2}{3}\frac{1}{3}\frac{2}{3} = \frac{4}{81}$	$\frac{1}{324}$

Total probability $\sum = \frac{77}{10800} \approx 0.007$



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

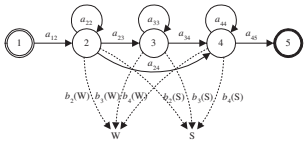
$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

All the alternative paths: the likelihood

Given the output sequence $\mathbf{x} =$ W, W, S, W.

$\mathbf{q} = 1, q_1, q_2, q_3, q_4, 5$	$\prod_t a_{q_{t-1}q_t}$	$\prod_t b_{q_t}(x_t)$	$\prod_t b_{q_t}(x_t)\, a_{q_{t-1}q_t}$
1, 2, 2, 2, 4, 5	$1\ \frac{1}{4}\frac{1}{4}\frac{1}{2}\frac{1}{2} = \frac{1}{64}$	$\frac{1}{3}\frac{1}{3}\frac{2}{3}\frac{2}{3} = \frac{4}{81}$	$\frac{1}{1296}$
1, 2, 2, 3, 4, 5	$1\ \frac{1}{4}\frac{1}{4}\frac{1}{2}\frac{1}{2} = \frac{1}{64}$	$\frac{1}{3}\frac{1}{3}\frac{4}{5}\frac{2}{3} = \frac{8}{135}$	$\frac{1}{1080}$
1, 2, 2, 4, 4, 5	$1\ \frac{1}{4}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{32}$	$\frac{1}{3}\frac{1}{3}\frac{1}{3}\frac{2}{3} = \frac{2}{81}$	$\frac{1}{1296}$
1, 2, 3, 3, 4, 5	$1\ \frac{1}{4}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{32}$	$\frac{1}{3}\frac{1}{5}\frac{4}{5}\frac{2}{3} = \frac{8}{225}$	$\frac{1}{900}$
1, 2, 3, 4, 4, 5	$1\ \frac{1}{4}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{32}$	$\frac{1}{3}\frac{1}{5}\frac{1}{3}\frac{2}{3} = \frac{2}{135}$	$\frac{1}{2160}$
1, 2, 4, 4, 4, 5	$1\ \frac{1}{2}\frac{1}{2}\frac{1}{2}\frac{1}{2} = \frac{1}{16}$	$\frac{1}{3}\frac{2}{3}\frac{1}{3}\frac{2}{3} = \frac{4}{81}$	$\frac{1}{324}$

Total probability $\sum = \frac{77}{10800} \approx 0.007$



$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

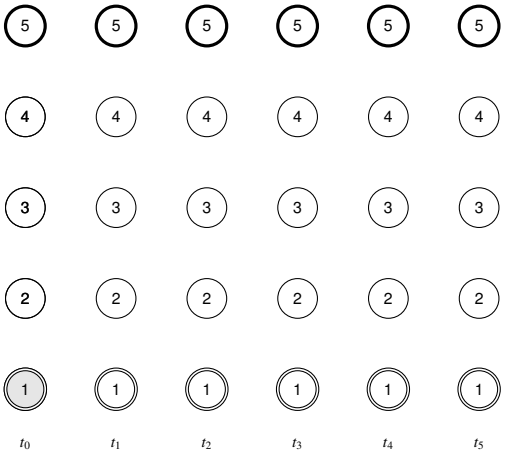
$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$$\Pr(\mathbf{x}\,|A, B) = \sum_{\forall \mathbf{q}} \prod_t b_{q_t}(x_t)\, a_{q_{t-1}q_t}$$

Trellis: the most probable path

t_0 t_1 t_2 t_3 t_4 t_5

Trellis: the most probable path

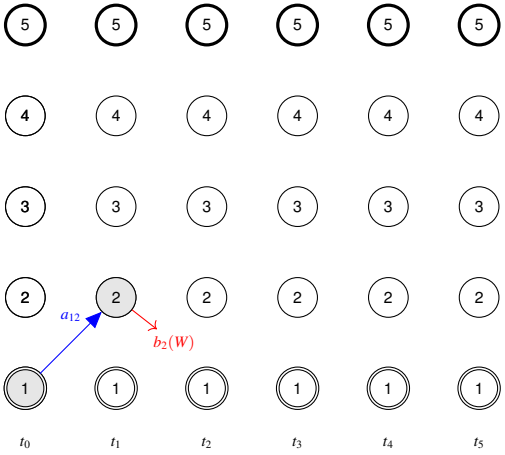


$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

Trellis: the most probable path



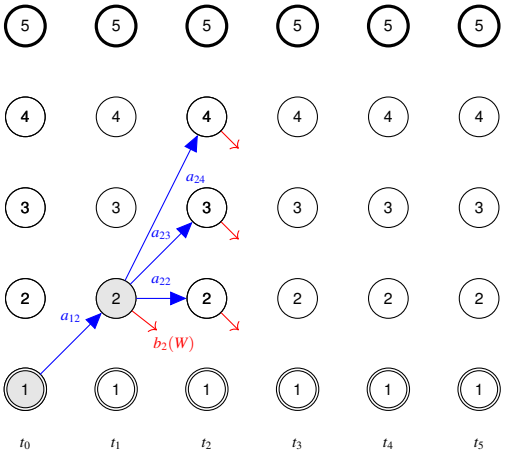
$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$p_{12} = \frac{1}{3}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

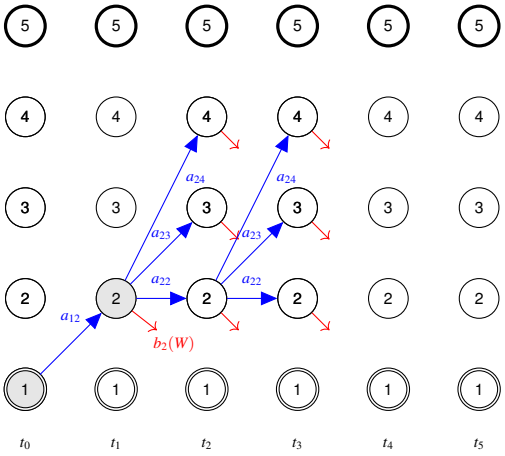
$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

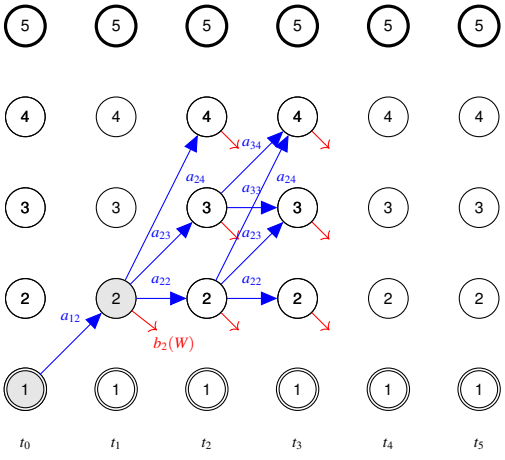
$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$

Trellis: the most probable path



$\mathbf{x}$ = W, W, S, W

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

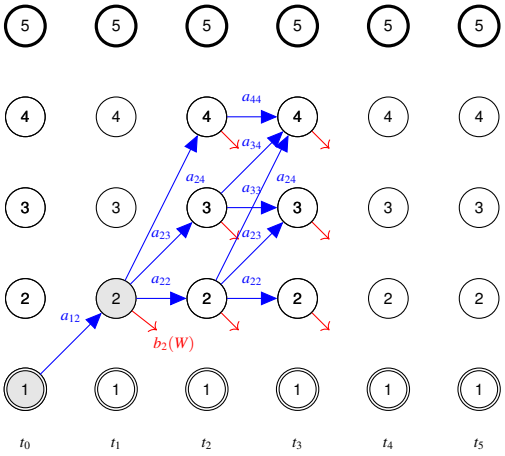
$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36}$, $p_{123} = \frac{1}{60}$, $p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216}$, $p_{1223} = \frac{1}{180}$, $p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600}$, $p_{1234} = \frac{1}{360}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

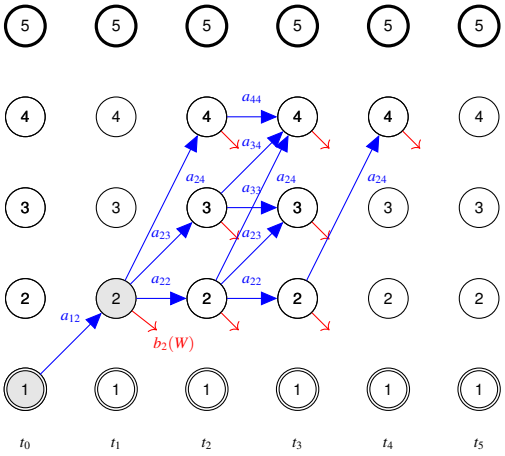
$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36}, \; p_{123} = \frac{1}{60}, \; p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216}, \; p_{1223} = \frac{1}{180}, \; p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600}, \; p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

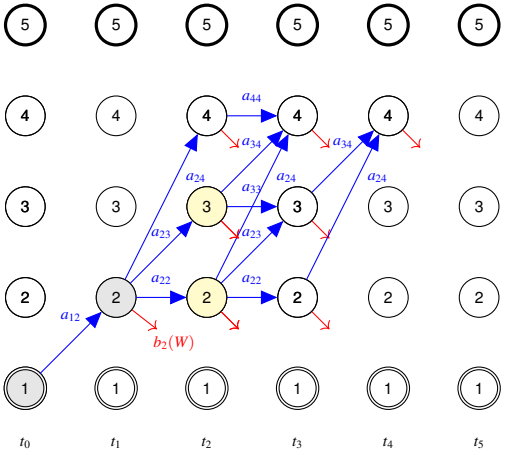
$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600},\ p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

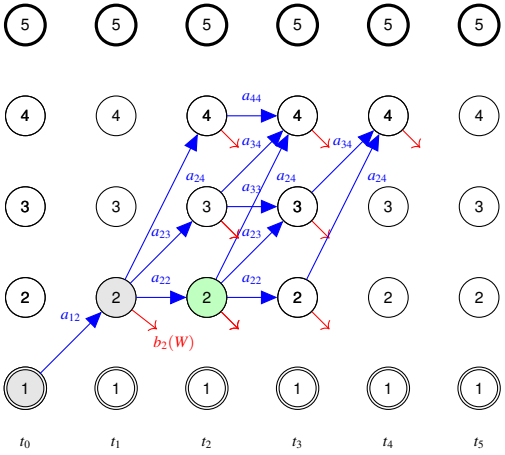
$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600},\ p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$
$p_{12?34} = \max\{p_{1223}, p_{1233}\}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

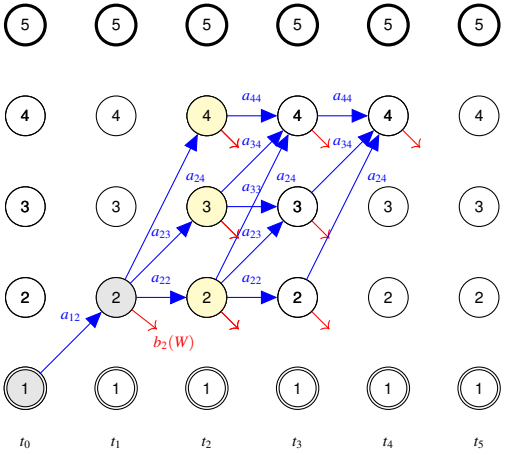
$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600},\ p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$
$p_{12?34} = p_{12234} = \frac{1}{2160}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

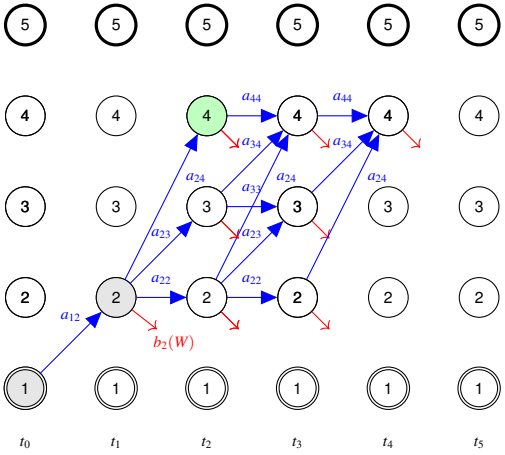
$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600},\ p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$
$p_{12?34} = p_{12234} = \frac{1}{2160}$
$p_{12?44} = \max\{p_{12244}, p_{12344}, p_{12444}\}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

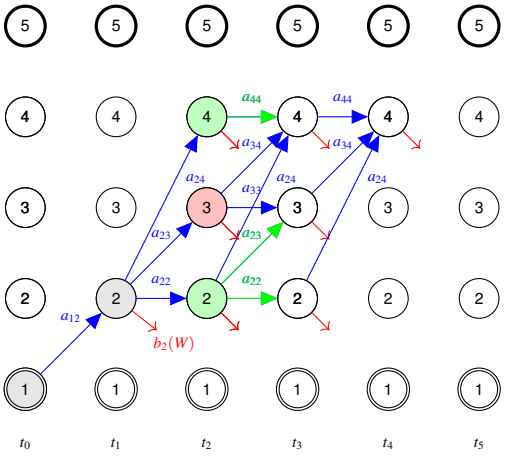
$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600},\ p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$
$p_{12?34} = p_{12234} = \frac{1}{2160}$
$p_{12?44} = p_{12444} = \frac{1}{162}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

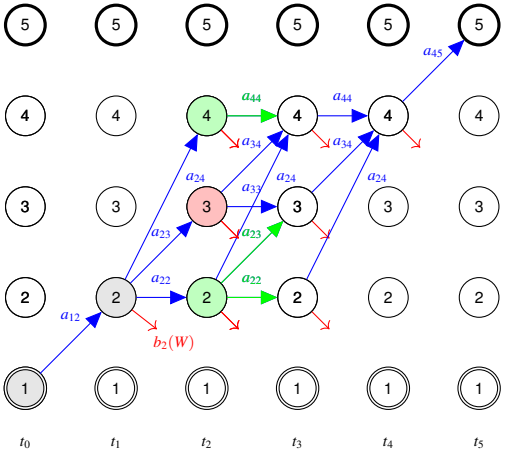
$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600},\ p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$
$p_{12?34} = p_{12234} = \frac{1}{2160}$
$p_{12?44} = p_{12444} = \frac{1}{162}$

Trellis: the most probable path



$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$p_{12} = \frac{1}{3}$

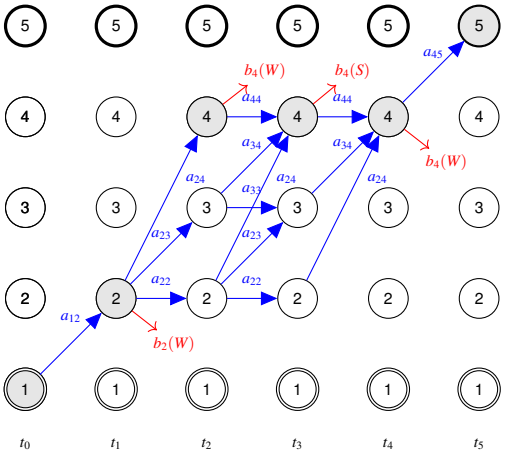
$p_{122} = \frac{1}{36},\ p_{123} = \frac{1}{60},\ p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216},\ p_{1223} = \frac{1}{180},\ p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600},\ p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$
$p_{12?34} = p_{12234} = \frac{1}{2160}$
$p_{12?44} = p_{12444} = \frac{1}{162}$

$p_{12??45} = \max\{p_{122245}, p_{122345}, p_{124445}\}$

Trellis: the most probable path



Viterbi

$\mathbf{x} = \text{W, W, S, W}$

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 1/4 & 1/4 & 1/2 & 0 \\ 0 & 0 & 1/2 & 1/2 & 0 \\ 0 & 0 & 0 & 1/2 & 1/2 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

$$B = \begin{bmatrix} 0 & 1/3 & 1/5 & 2/3 & 0 \\ 0 & 2/3 & 4/5 & 1/3 & 0 \end{bmatrix}$$

$p_{12} = \frac{1}{3}$

$p_{122} = \frac{1}{36}$, $p_{123} = \frac{1}{60}$, $p_{124} = \frac{1}{9}$

$p_{1222} = \frac{1}{216}$, $p_{1223} = \frac{1}{180}$, $p_{1224} = \frac{1}{216}$
$p_{1233} = \frac{1}{600}$, $p_{1234} = \frac{1}{360}$
$p_{1244} = \frac{1}{54}$

$p_{12224} = \frac{1}{1296}$
$p_{12?34} = p_{12234} = \frac{1}{2160}$
$p_{12?44} = p_{12444} = \frac{1}{162}$

$p_{12??45} = p_{124445} = \frac{1}{324}$

sum vs max

sum vs max

$$\Pr(\mathbf{x}|A,B) = \sum_{\forall \mathbf{q}} \prod_{t} b_{q_t}(x_t)\, a_{q_{t-1}q_t} = \frac{77}{10800} \approx 0.007$$

sum vs max

$$\Pr(\mathbf{x}|A,B) = \sum_{\forall \mathbf{q}} \prod_{t} b_{q_t}(x_t)\, a_{q_{t-1}q_t} = \frac{77}{10800} \approx 0.007$$

$$\Pr(\mathbf{x}|A,B) \approx \max_{\forall \mathbf{q}} \prod_{t} b_{q_t}(x_t)\, a_{q_{t-1}q_t} = \frac{1}{324} \approx 0.003$$

Viterbi recursion and backtracking

Viterbi recursion and backtracking

We define the *best path probability* as the probability of the most likely state sequence that generates the observations up to time t, $\mathbf{X}^t$, and ends in state j:

$$\lambda_t(j) \triangleq \max_{\forall \mathbf{q}^{t-1}} \left\{ \Pr\left(\mathbf{q}^{t-1}, q_t = j, \mathbf{X}^t | \Theta \right) \right\}; \quad \forall j \in \mathcal{Q}; \quad \lambda_0(j) = 1 \; \forall j \in \mathcal{Q}$$

Viterbi recursion and backtracking

We define the *best path probability* as the probability of the most likely state sequence that generates the observations up to time t, $\mathbf{X}^t$, and ends in state j:

$$\lambda_t(j) \triangleq \max_{\forall \mathbf{q}^{t-1}} \left\{ \Pr\left(\mathbf{q}^{t-1}, q_t = j, \mathbf{X}^t \,|\Theta \right) \right\}; \quad \forall j \in \mathcal{Q}; \quad \lambda_0(j) = 1 \; \forall j \in \mathcal{Q}$$

By induction, we have

$$\lambda_t\left(j\right) = \max_{\forall i \in \mathcal{Q}} \left\{ \lambda_{t-1}(i) a_{ij} \right\} b_j\left(x_t\right)$$

Viterbi recursion and backtracking

We define the *best path probability* as the probability of the most likely state sequence that generates the observations up to time t, $\mathbf{X}^t$, and ends in state j:

$$\lambda_t(j) \triangleq \max_{\forall \mathbf{q}^{t-1}} \left\{ \Pr\left(\mathbf{q}^{t-1}, q_t = j, \mathbf{X}^t | \Theta \right) \right\}; \quad \forall j \in \mathcal{Q}; \quad \lambda_0(j) = 1 \; \forall j \in \mathcal{Q}$$

By induction, we have

$$\lambda_t(j) = \max_{\forall i \in \mathcal{Q}} \{\lambda_{t-1}(i) a_{ij}\} b_j(x_t)$$

To recover the best sequence of hidden states, we define

$$\xi_t(j) \triangleq \arg \max_{\forall i \in \mathcal{Q}} \{\lambda_{t-1}(i) a_{ij}\}$$

and, starting from $\tilde{q}_T = \arg \max_{\forall i \in \mathcal{Q}} \{\lambda_T(i)\}$, apply the backward recursion

$$\tilde{q}_t = \xi_{t+1}(\tilde{q}_{t+1}); \quad t = T-1, T-2, \ldots, 1$$

Estimating the parameters: the intuitive way

How can we estimate the transition probabilities A in the *hidden* model?

Estimating the parameters: the intuitive way

How can we estimate the transition probabilities A in the *hidden* model?

1. Find the most probable sequence of hidden states (Viterbi)
2. Count the transitions from i to j, and the visits to i: $\gamma(i,j)$ and $\gamma(i)$
3. Estimate $\tilde{a}_{ij} = \frac{\gamma(i,j)}{\gamma(i)}$

Estimating the parameters: the intuitive way

How can we estimate the transition probabilities A in the *hidden* model?

1. Find the most probable sequence of hidden states (Viterbi)
2. Count the transitions from i to j, and the visits to i: $\gamma(i,j)$ and $\gamma(i)$
3. Estimate $\tilde{a}_{ij} = \frac{\gamma(i,j)}{\gamma(i)}$

And the emission probabilities B?

Estimating the parameters: the intuitive way

How can we estimate the transition probabilities A in the *hidden* model?

1. Find the most probable sequence of hidden states (Viterbi)
2. Count the transitions from i to j, and the visits to i: $\gamma(i,j)$ and $\gamma(i)$
3. Estimate $\tilde{a}_{ij} = \frac{\gamma(i,j)}{\gamma(i)}$

And the emission probabilities B?

1. Find the most probable sequence of hidden states (Viterbi)
2. Count the times the model was in state j and emitted x: $\delta_j(x)$
3. Count the times the model was in state j: $\gamma(j)$
4. Estimate $\tilde{b}_j(x) = \frac{\delta_j(x)}{\gamma(j)}$

HMM: formal definition

HMM: formal definition

To model a sequence $\mathbf{X} = x_1, x_2, \ldots, x_T$, a hidden Markov model can be defined by the structure

$$\Theta = \langle \mathcal{Q}, \mathbf{A}, \boldsymbol{\pi}, \mathcal{B} \rangle,$$

HMM: formal definition

To model a sequence $\mathbf{X} = x_1, x_2, \ldots, x_T$, a hidden Markov model can be defined by the structure

$$\Theta = \langle \mathcal{Q}, \mathbf{A}, \boldsymbol{\pi}, \mathcal{B} \rangle,$$

where:

i) $\mathcal{Q} = \{Q \in [1 \ldots N_Q]\}$ is the set of states.

ii) $\mathbf{A} = [a_{ij} = \Pr(Q_t = j \,|Q_{t-1} = i)], \forall i,j \in \mathcal{Q}$, is the matrix of transition probabilities, where $Q_t \in \mathcal{Q}$ is the model state at time $t \in [1 \ldots T]$, $a_{ij} \geq 0 \ \forall i,j$ and $\sum_j a_{ij} \stackrel{\circ}{=} 1 \ \forall i$.

iii) $\boldsymbol{\pi} = [\pi_j = \Pr(Q_1 = j)]$ is the initial state probability vector. In the case of left-to-right HMM this vector is $\boldsymbol{\pi} = \boldsymbol{\delta}_1$.

iv) $\mathcal{B} = \{b_j(x_t) = \Pr(X_t = x_t \,|Q_t = j), \forall j \in \mathcal{Q}\}$, is the set of observation (or emission) probability distributions,

—∘ where $x_t \in \mathcal{X} = \{X \in [1 \ldots N_x]\}$, the set of observable (discrete) *symbols*.

HMM: hypotheses

HMM: hypotheses

The *first-order Markov* hypothesis: given the present state, the past has no influence on the future evolution of the chain.
For a discrete-time first-order Markov chain, the dependence is truncated to

$$\Pr\left(Q_t = j \,|Q_{t-1} = i, Q_{t-2} = k, \ldots\right) = \Pr\left(Q_t = j \,|Q_{t-1} = i\right)$$

HMM: hypotheses

The *first-order Markov* hypothesis: given the present state, the past has no influence on the future evolution of the chain.
For a discrete-time first-order Markov chain, the dependence is truncated to

$$\Pr\left(Q_t = j \,|Q_{t-1} = i, Q_{t-2} = k, \dots\right) = \Pr\left(Q_t = j \,|Q_{t-1} = i\right)$$

The *output independence* hypothesis: given the current state, the observation depends neither on the past states nor on the past observations.
Given the state sequence $\mathbf{q}$,

$$\Pr\left(\mathbf{X} \,|\mathbf{q}\right) = \prod_t \Pr\left(X_t = x_t \,|\mathbf{q}\right) = \prod_t \Pr\left(X_t = x_t \,|Q_t = q_t\right)$$

Likelihood and the auxiliary function

Likelihood and the auxiliary function

The HMM likelihood can be defined as the probability of the observed data given the model:

$$
\begin{aligned}
\mathcal{L}_\Theta(\mathbf{X}) = \sum_{\forall \mathbf{q}} \mathcal{L}_\Theta(\mathbf{X}, \mathbf{q}) &\triangleq \sum_{\forall \mathbf{q}} \Pr(\mathbf{X}, \mathbf{q} \,|\Theta) \\
&= \sum_{\forall \mathbf{q}} \{\Pr(\mathbf{X} \,|\mathbf{q}, \Theta) \Pr(\mathbf{q} \,|\Theta)\} \\
&= \sum_{\forall \mathbf{q}} \left\{ \prod_t \Pr(x_t \,|q_t, \Theta) \prod_{t=2} \Pr(q_t \,|q_{t-1}, \Theta) \right\} \\
&= \sum_{\forall \mathbf{q}} \left\{ \prod_t b_{q_t}(x_t) \prod_{t=2}^{T} a_{q_{t-1} q_t} \right\} \\
&= \sum_{\forall \mathbf{q}} \prod_t a_{q_{t-1} q_t} b_{q_t}(x_t)
\end{aligned}
$$

where $a_{01} = \pi_1 = 1$.

Likelihood and the auxiliary function

We define the auxiliary function from the log-likelihood:

$$\mathcal{O}(\Theta, \tilde{\Theta}) \triangleq \frac{1}{\Pr(\mathbf{X}|\Theta)} \sum_{\forall \mathbf{q}} \Pr(\mathbf{X}, \mathbf{q}|\Theta) \log \Pr(\mathbf{X}, \mathbf{q}|\tilde{\Theta})$$

Likelihood and the auxiliary function

We define the auxiliary function from the log-likelihood:

$$\mathcal{O}(\Theta, \tilde{\Theta}) \triangleq \frac{1}{\Pr(\mathbf{X}|\Theta)} \sum_{\forall \mathbf{q}} \Pr(\mathbf{X}, \mathbf{q}|\Theta) \log \Pr(\mathbf{X}, \mathbf{q}|\tilde{\Theta})$$

In the discrete case, from

$$\Pr\left(\mathbf{X}, \mathbf{q}\,|\Theta\right) = \prod_t b_{q_t}\left(x_t\right) a_{q_{t-1}q_t}$$

we have

$$\log \Pr\left(\mathbf{X}, \mathbf{q}\,\middle|\tilde{\Theta}\right) = \sum_t \log \tilde{b}_{q_t}\left(x_t\right) + \sum_t \log \tilde{a}_{q_{t-1}q_t}$$

Maximizing the auxiliary function

Maximizing the auxiliary function

The auxiliary function splits into two independent terms, one per parameter set:

$$\mathcal{O}(\Theta, \tilde{\Theta}) = \mathcal{O}_b(\Theta, \tilde{b}_j) + \mathcal{O}_a(\Theta, \tilde{a}_{ij})$$

$$\mathcal{O}_a(\Theta, \tilde{a}_{ij}) = \sum_i \sum_j \sum_t \Pr\left(q_{t-1} = i, q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{a}_{ij}$$

$$\mathcal{O}_b(\Theta, \tilde{b}_j) = \sum_j \sum_t \Pr\left(q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{b}_j\,(x_t)$$

Maximizing the auxiliary function

The auxiliary function splits into two independent terms, one per parameter set:

$$\mathcal{O}(\Theta, \tilde{\Theta}) = \mathcal{O}_b(\Theta, \tilde{b}_j) + \mathcal{O}_a(\Theta, \tilde{a}_{ij})$$

$$\mathcal{O}_a(\Theta, \tilde{a}_{ij}) = \sum_i \sum_j \sum_t \Pr\left(q_{t-1} = i, q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{a}_{ij}$$
$$\mathcal{O}_b(\Theta, \tilde{b}_j) = \sum_j \sum_t \Pr\left(q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{b}_j\,(x_t)$$

We maximize it subject to the normalization constraints $\sum_j \tilde{a}_{ij} \stackrel{\circ}{=} 1 \;\; \forall i \in \mathcal{Q}$, and $\sum_k \tilde{b}_j(x_k) \stackrel{\circ}{=} 1 \;\; \forall j \in \mathcal{Q}$

Maximizing the auxiliary function

The auxiliary function splits into two independent terms, one per parameter set:

$$\mathcal{O}(\Theta, \tilde{\Theta}) = \mathcal{O}_b(\Theta, \tilde{b}_j) + \mathcal{O}_a(\Theta, \tilde{a}_{ij})$$

$$\mathcal{O}_a(\Theta, \tilde{a}_{ij}) = \sum_i \sum_j \sum_t \Pr\left(q_{t-1} = i, q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{a}_{ij}$$
$$\mathcal{O}_b(\Theta, \tilde{b}_j) = \sum_j \sum_t \Pr\left(q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{b}_j\,(x_t)$$

We maximize it subject to the normalization constraints $\sum_j \tilde{a}_{ij} \doteq 1 \;\; \forall i \in \mathcal{Q}$, and $\sum_k \tilde{b}_j(x_k) \doteq 1 \;\; \forall j \in \mathcal{Q}$ so we use Lagrange multipliers

$$\nabla_{\tilde{a}} \left(\mathcal{O}_a(\Theta, \tilde{a}_{ij}) - \sum_i \ell_i \left(\sum_j \tilde{a}_{ij} - 1 \right) \right) = 0$$

Maximizing the auxiliary function

The auxiliary function splits into two independent terms, one per parameter set:

$$\mathcal{O}(\Theta, \tilde{\Theta}) = \mathcal{O}_b(\Theta, \tilde{b}_j) + \mathcal{O}_a(\Theta, \tilde{a}_{ij})$$

$$\mathcal{O}_a(\Theta, \tilde{a}_{ij}) = \sum_i \sum_j \sum_t \Pr\left(q_{t-1} = i, q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{a}_{ij}$$
$$\mathcal{O}_b(\Theta, \tilde{b}_j) = \sum_j \sum_t \Pr\left(q_t = j \,|\mathbf{X}, \Theta\right) \log \tilde{b}_j\,(x_t)$$

We maximize it subject to the normalization constraints $\sum_j \tilde{a}_{ij} \stackrel{\circ}{=} 1 \;\; \forall i \in \mathcal{Q}$, and $\sum_k \tilde{b}_j(x_k) \stackrel{\circ}{=} 1 \;\; \forall j \in \mathcal{Q}$ so we use Lagrange multipliers

$$\nabla_{\tilde{a}} \left(\mathcal{O}_a(\Theta, \tilde{a}_{ij}) - \sum_i \ell_i \left(\sum_j \tilde{a}_{ij} - 1 \right) \right) = 0$$

—∘ the same holds for $\tilde{b}_j(x_k)$.

Reestimation formulas (discrete case)

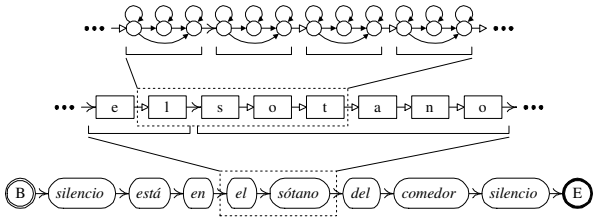
$$\tilde{a}_{ij} = \frac{\sum_t \frac{\Pr(\mathbf{X}, q_{t-1} = i, q_t = j \,|\Theta)}{\Pr(\mathbf{X}\,|\Theta)}}{\sum_t \frac{\Pr(\mathbf{X}, q_{t-1} = i \,|\Theta)}{\Pr(\mathbf{X}\,|\Theta)}} = \frac{\sum_t \gamma_t(i,j)}{\sum_t \gamma_t(i)}$$

$$\tilde{b}_j(x_k) = \frac{\sum_t \frac{\Pr(\mathbf{X}, q_t = j \,|\Theta)}{\Pr(\mathbf{X}\,|\Theta)} 1_k(x_t)}{\sum_t \frac{\Pr(\mathbf{X}, q_t = j \,|\Theta)}{\Pr(\mathbf{X}\,|\Theta)}} = \frac{\sum_{\forall t / x_t = x_k} \gamma_t(j)}{\sum_t \gamma_t(j)}$$

where
$1_k(x_t) = 1$ if $x_t = x_k$ and 0 otherwise,
$\gamma_t(j) \triangleq \Pr(q_t = j \,|\mathbf{X}, \Theta)$, and
$\gamma_t(i,j) \triangleq \Pr(q_{t-1} = i, q_t = j \,|\mathbf{X}, \Theta)$.
$\gamma_t(j)$ and $\gamma_t(i,j)$ are computed in $O(N^2 T)$ with the forward-backward algorithm, without enumerating all paths.

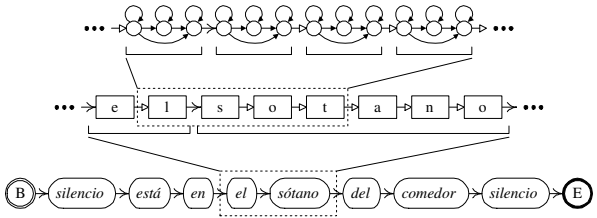
(not large) Language models

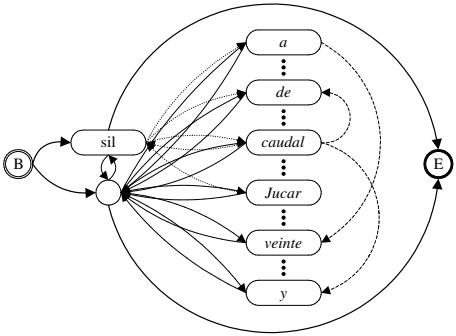
(not large) Language models



- A word is a chain of phoneme HMMs, a sentence is a chain of words

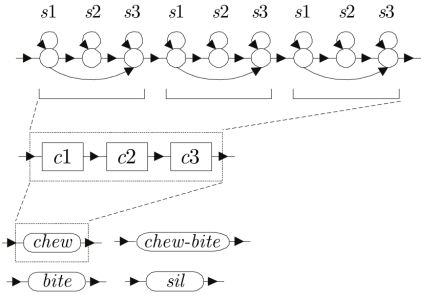
(not large) Language models





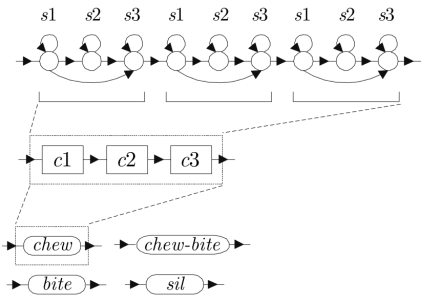
- A word is a chain of phoneme HMMs, a sentence is a chain of words
- The language model is another automaton: states are words, and it loops
- Decoding is still Viterbi, over the whole composed model

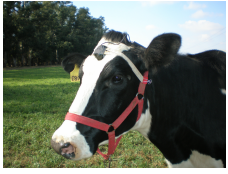
(not large, not human) Language models

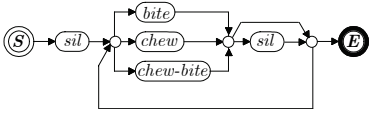


- Same construction, new vocabulary: bite, chew, chew-bite

(not large, not human) Language models

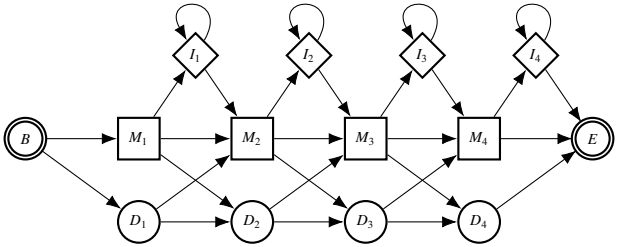






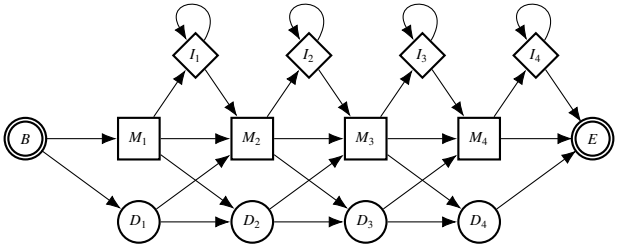
- Same construction, new vocabulary: bite, chew, chew-bite
- The grammar says how a cow grazes
- Same algebra, no speech at all

(not large, not even a sound) HMMs for biological sequences



- One model per family: the topology is its multiple alignment
- A different question: does this sequence belong to the family?
- Nothing new: same Viterbi, same recursions, same reestimation

(not large, not even a sound) HMMs for biological sequences



- One model per family: the topology is its multiple alignment
- A different question: does this sequence belong to the family?
- Nothing new: same Viterbi, same recursions, same reestimation
- Pfam: 30,134 families, searched with HMMER
- Homology and learning at the same time

From generative to discriminative

From generative to discriminative

- An HMM models $\Pr(\mathbf{X}, \mathbf{q})$: it spends capacity explaining the input
  $\Rightarrow$ it must assign probability to every sequence that could occur

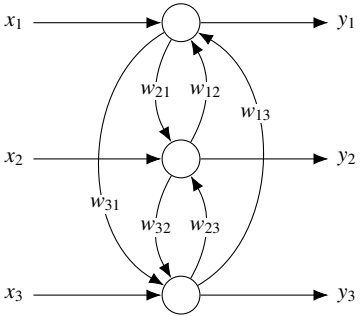
From generative to discriminative

- An HMM models $\Pr(\mathbf{X}, \mathbf{q})$: it spends capacity explaining the input
  $\Rightarrow$ it must assign probability to every sequence that could occur
- A Conditional Random Field (CRF) models $\Pr(\mathbf{q} \,|\mathbf{X})$ directly
  $\multimap$ same state graph, same Viterbi decoding — what changes is what is optimized
- The input is conditioned on, not generated
  $\Rightarrow$ features can look anywhere in $\mathbf{X}$: windows, overlapping k-mers, physicochemical properties
- Used as the output layer of deep models
  $\multimap$ SignalP 6.0: a protein language model with a CRF decoder

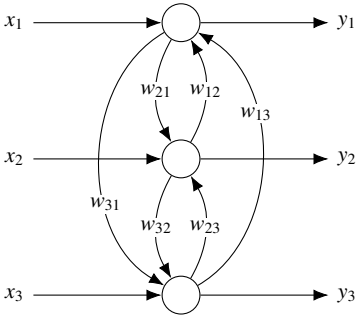
Recurrent neural networks

(RNN)

Hopfield: architecture and Hebbian learning

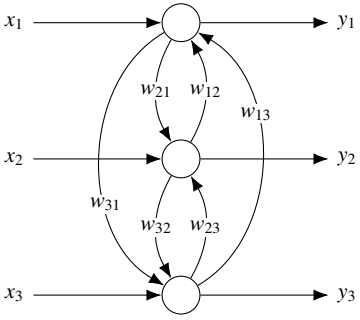


Hopfield: architecture and Hebbian learning



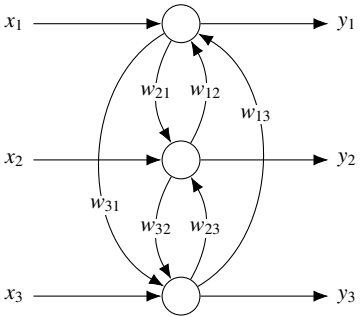
- Fully connected, symmetric, no self-connections
  $w_{ji} = w_{ij} \quad \forall\, i \neq j, \quad w_{ii} = 0 \quad \forall i$

Hopfield: architecture and Hebbian learning



- Fully connected, symmetric, no self-connections
  $w_{ji} = w_{ij} \quad \forall\, i \neq j, \quad w_{ii} = 0 \quad \forall i$
- Hebbian storage, in one shot (no iteration):
  $$w_{ji} = \frac{1}{N} \sum_{k=1}^{P} x_{kj}^* x_{ki}^*$$
  —○ for P fundamental memories (training patterns) $X^* = \{\mathbf{x}_k^* \in \mathbb{R}^N\}$

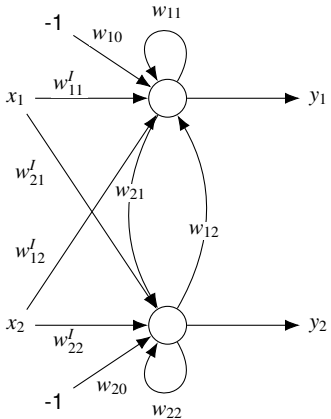
Hopfield: architecture and Hebbian learning



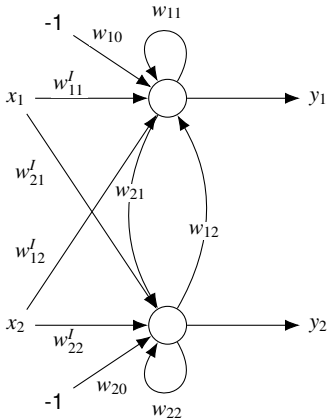
- Fully connected, symmetric, no self-connections
  $w_{ji} = w_{ij} \quad \forall\, i \neq j, \quad w_{ii} = 0 \quad \forall i$
- Hebbian storage, in one shot (no iteration):
  $$w_{ji} = \frac{1}{N} \sum_{k=1}^{P} x_{kj}^* x_{ki}^*$$
  —○ for P fundamental memories (training patterns) $X^* = \{\mathbf{x}_k^* \in \mathbb{R}^N\}$
- Retrieval is iterative: from $\mathbf{y}(0) = \mathbf{x}$,
  $$y_{j^*}(n) = sgn\left(\sum_{i=1}^{N} w_{j^*i}\, y_i(n-1)\right)$$
  —○ for a random unit j^*, until no output changes

A recurrent network and its forward pass

A recurrent network and its forward pass



A recurrent network and its forward pass

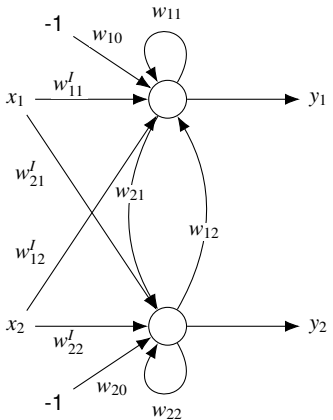


For each neuron j and each input i

$$y_{t,j} = \phi(v_{t,j}) = \phi\left(\sum_i w^I_{ji} x_{t,i} + \sum_i w_{ji} y_{t-1,i}\right)$$

where W^I and W are the weight matrices, and $\phi(v) = \frac{2}{1+e^{-v}} - 1$ the symmetric sigmoid activation.

A recurrent network and its forward pass



For each neuron j and each input i

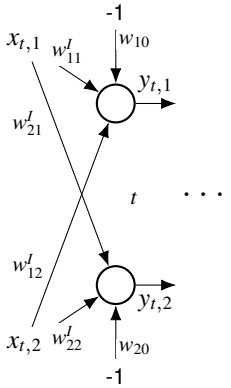
$$y_{t,j} = \phi(v_{t,j}) = \phi\left(\sum_i w^I_{ji} x_{t,i} + \sum_i w_{ji} y_{t-1,i}\right)$$

where W^I and W are the weight matrices, and $\phi(v) = \frac{2}{1+e^{-v}} - 1$ the symmetric sigmoid activation.

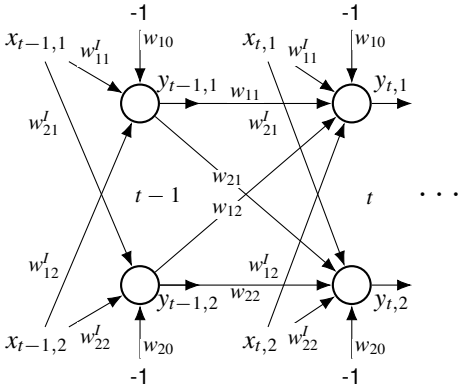
The forward pass is simply:

for $t = 0$ **to** $T - 1$
$\quad \mathbf{y}_t \leftarrow \phi\left(W^I \mathbf{x}_t + W \mathbf{y}_{t-1}\right)$

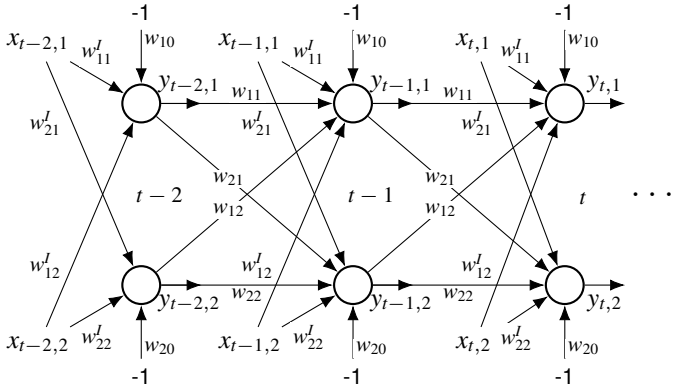
Backpropagation through time: unrolled in time



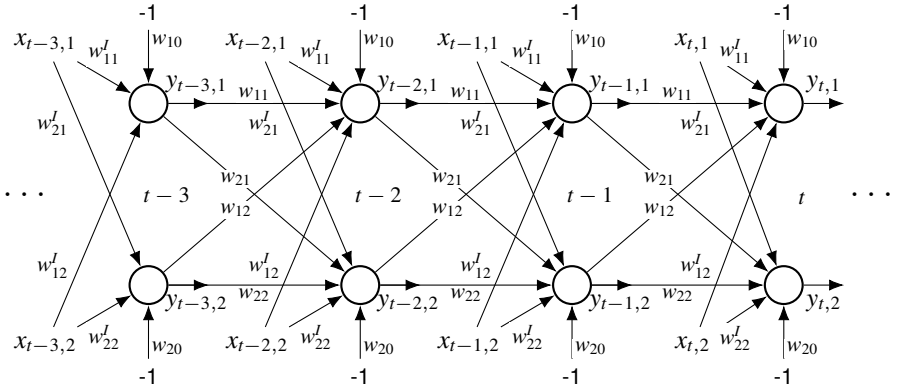
Backpropagation through time: unrolled in time



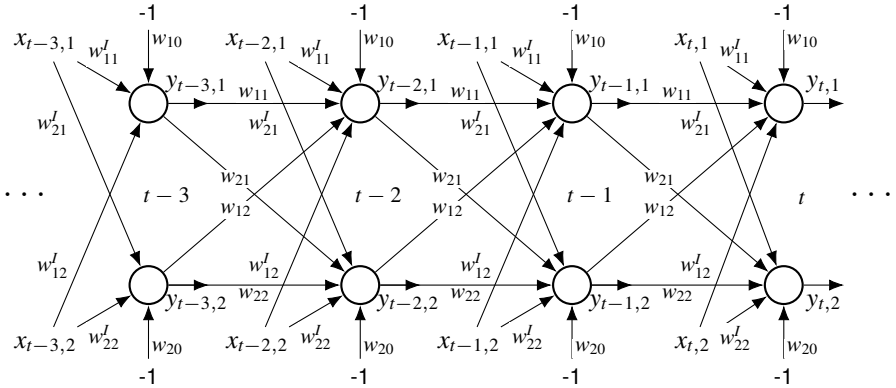
Backpropagation through time: unrolled in time



Backpropagation through time: unrolled in time



Backpropagation through time: unrolled in time



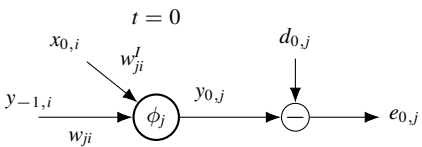
The total error is $E = \sum_{t=0}^{T} E_t = \frac{1}{2} \sum_{t=0}^{T} \sum_{k=1}^{N} e_{t,k}^2 = \frac{1}{2} \sum_{t=0}^{T} \sum_{k=1}^{N} (y_{t,k} - d_{t,k})^2$

Gradients for the recurrent weights

To train the recurrent weights W, for the total error we have $\frac{\partial E}{\partial w_{ji}} = \frac{\partial E_0}{\partial w_{ji}} + \frac{\partial E_1}{\partial w_{ji}} + \cdots + \frac{\partial E_T}{\partial w_{ji}}$

Gradients for the recurrent weights

To train the recurrent weights W, for the total error we have $\frac{\partial E}{\partial w_{ji}} = \frac{\partial E_0}{\partial w_{ji}} + \frac{\partial E_1}{\partial w_{ji}} + \cdots + \frac{\partial E_T}{\partial w_{ji}}$

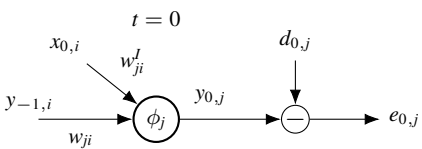


Unrolled in one layer, from $\mathbf{y}_{-1}$

$$E_0 = \frac{1}{2}\sum_{k=1}^{N}(y_{0,k} - d_{0,k})^2$$

Gradients for the recurrent weights

To train the recurrent weights W, for the total error we have $\frac{\partial E}{\partial w_{ji}} = \frac{\partial E_0}{\partial w_{ji}} + \frac{\partial E_1}{\partial w_{ji}} + \cdots + \frac{\partial E_T}{\partial w_{ji}}$



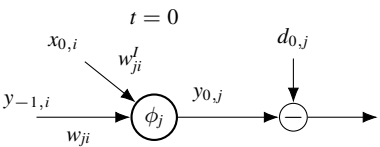
$$\frac{\partial E_0}{\partial w_{ji}} = \frac{\partial E_0}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} \frac{\partial v_{0,j}}{\partial w_{ji}}$$

Unrolled in one layer, from $\mathbf{y}_{-1}$

$$E_0 = \frac{1}{2} \sum_{k=1}^{N} (y_{0,k} - d_{0,k})^2$$

Gradients for the recurrent weights

To train the recurrent weights W, for the total error we have $\dfrac{\partial E}{\partial w_{ji}} = \dfrac{\partial E_0}{\partial w_{ji}} + \dfrac{\partial E_1}{\partial w_{ji}} + \cdots + \dfrac{\partial E_T}{\partial w_{ji}}$



$$\frac{\partial E_0}{\partial w_{ji}} = \frac{\partial E_0}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} \frac{\partial v_{0,j}}{\partial w_{ji}}$$

$$\frac{\partial E_0}{\partial y_{0,j}} = \frac{\partial}{\partial y_{0,j}} \frac{1}{2} \sum_{k=1}^{N} e_{0,k}^2 = \sum_{k=1}^{N} e_{0,k} \frac{\partial e_{0,k}}{\partial y_{0,j}} = e_{0,j} = y_{0,j} - d_{0,j}$$

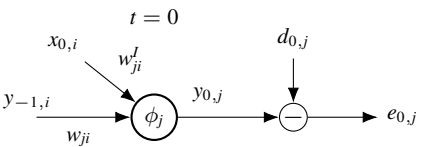
$$\frac{\partial y_{0,j}}{\partial v_{0,j}} = (1 - y_{0,j})(1 + y_{0,j}); \quad \frac{\partial v_{0,j}}{\partial w_{ji}} = \frac{\partial}{\partial w_{ji}} \sum_{\ell} w_{j\ell} y_{-1,\ell} = y_{-1,i}$$

Unrolled in one layer, from $\mathbf{y}_{-1}$

$$E_0 = \frac{1}{2} \sum_{k=1}^{N} (y_{0,k} - d_{0,k})^2$$

Gradients for the recurrent weights

To train the recurrent weights W, for the total error we have $\dfrac{\partial E}{\partial w_{ji}} = \dfrac{\partial E_0}{\partial w_{ji}} + \dfrac{\partial E_1}{\partial w_{ji}} + \cdots + \dfrac{\partial E_T}{\partial w_{ji}}$



Unrolled in one layer, from $\mathbf{y}_{-1}$

$$E_0 = \frac{1}{2}\sum_{k=1}^{N}(y_{0,k} - d_{0,k})^2$$

$$\frac{\partial E_0}{\partial w_{ji}} = \frac{\partial E_0}{\partial y_{0,j}}\frac{\partial y_{0,j}}{\partial v_{0,j}}\frac{\partial v_{0,j}}{\partial w_{ji}}$$

$$\frac{\partial E_0}{\partial y_{0,j}} = \frac{\partial}{\partial y_{0,j}}\frac{1}{2}\sum_{k=1}^{N} e_{0,k}^2 = \sum_{k=1}^{N} e_{0,k}\frac{\partial e_{0,k}}{\partial y_{0,j}} = e_{0,j} = y_{0,j} - d_{0,j}$$

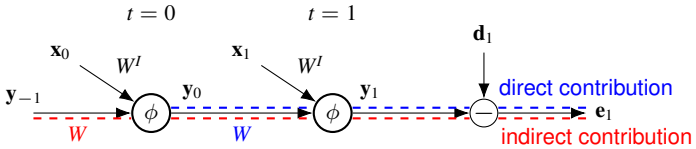
$$\frac{\partial y_{0,j}}{\partial v_{0,j}} = (1 - y_{0,j})(1 + y_{0,j}); \quad \frac{\partial v_{0,j}}{\partial w_{ji}} = \frac{\partial}{\partial w_{ji}}\sum_{\ell} w_{j\ell} y_{-1,\ell} = y_{-1,i}$$

$$\delta_{0,j} \triangleq \frac{\partial E_0}{\partial y_{0,j}}\frac{\partial y_{0,j}}{\partial v_{0,j}} = (y_{0,j} - d_{0,j})(1 - y_{0,j})(1 + y_{0,j})$$

$$\frac{\partial E_0}{\partial w_{ji}} = \delta_{0,j} y_{-1,i}$$

Each weight contributes twice

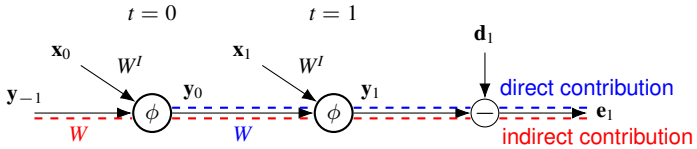
Unrolled in two layers, up to $t = 1$



$$\frac{\partial E_1}{\partial w_{ji}} = \frac{\partial E_1}{\partial w_{1,ji}} + \frac{\partial E_1}{\partial w_{0,ji}}$$

Each weight contributes twice

Unrolled in two layers, up to $t = 1$



$$\frac{\partial E_1}{\partial w_{ji}} = \frac{\partial E_1}{\partial w_{1,ji}} + \frac{\partial E_1}{\partial w_{0,ji}}$$

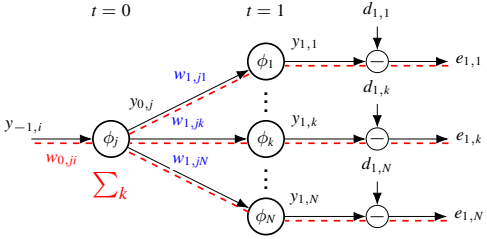
The direct contribution is

$$\begin{aligned}\frac{\partial E_1}{\partial w_{1,ji}} &= \frac{\partial E_1}{\partial y_{1,j}} \frac{\partial y_{1,j}}{\partial v_{1,j}} \frac{\partial v_{1,j}}{\partial w_{1,ji}} \\ &= e_{1,j} \cdot (1 - y_{1,j})(1 + y_{1,j}) \cdot y_{0,i} = \delta_{1,j}\, y_{0,i}\end{aligned}$$

The indirect path: previous output feeds every neuron

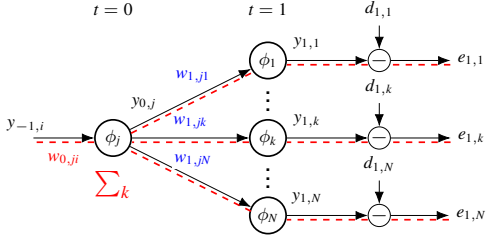
The indirect path: previous output feeds every neuron

The output $y_{0,j}$ at $t = 0$ feeds all the neurons k at $t = 1$:



The indirect path: previous output feeds every neuron

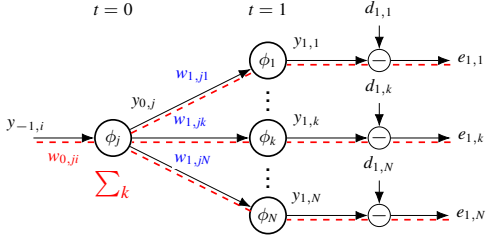
The output $y_{0,j}$ at $t = 0$ feeds all the neurons k at $t = 1$:



$$
\begin{aligned}
\frac{\partial E_1}{\partial w_{0,ji}} &= \frac{\partial E_1}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} \frac{\partial v_{0,j}}{\partial w_{0,ji}} \\
&= \frac{\partial}{\partial y_{0,j}} \frac{1}{2} \sum_{k=1}^{N} e_{1,k}^2 \cdot (1 - y_{0,j})(1 + y_{0,j}) \cdot y_{-1,i} \\
&= \sum_{k=1}^{N} e_{1,k} \frac{\partial e_{1,k}}{\partial y_{0,j}} \cdot (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i} \\
&= \sum_{k=1}^{N} e_{1,k} \frac{\partial e_{1,k}}{\partial y_{1,k}} \frac{\partial y_{1,k}}{\partial v_{1,k}} \frac{\partial v_{1,k}}{\partial y_{0,j}} \cdot (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i} \\
&= \sum_{k} e_{1,k} \cdot 1 \cdot (1 - y_{1,k})(1 + y_{1,k}) \cdot w_{1,jk} \cdot (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i} \\
&= \left(\sum_{k} \delta_{1,k} w_{1,jk} \right) (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i} \\
&= \delta_{0,j}\, y_{-1,i}
\end{aligned}
$$

The indirect path: previous output feeds every neuron

The output $y_{0,j}$ at $t = 0$ feeds all the neurons k at $t = 1$:



$$\frac{\partial E_1}{\partial w_{0,ji}} = \frac{\partial E_1}{\partial y_{0,j}} \frac{\partial y_{0,j}}{\partial v_{0,j}} \frac{\partial v_{0,j}}{\partial w_{0,ji}}$$

$$= \frac{\partial}{\partial y_{0,j}} \frac{1}{2} \sum_{k=1}^{N} e_{1,k}^2 \cdot (1 - y_{0,j})(1 + y_{0,j}) \cdot y_{-1,i}$$

$$= \sum_{k=1}^{N} e_{1,k} \frac{\partial e_{1,k}}{\partial y_{0,j}} \cdot (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i}$$

$$= \sum_{k=1}^{N} e_{1,k} \frac{\partial e_{1,k}}{\partial y_{1,k}} \frac{\partial y_{1,k}}{\partial v_{1,k}} \frac{\partial v_{1,k}}{\partial y_{0,j}} \cdot (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i}$$

$$= \sum_{k} e_{1,k} \cdot 1 \cdot (1 - y_{1,k})(1 + y_{1,k}) \cdot w_{1,jk} \cdot (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i}$$

$$= \left(\sum_{k} \delta_{1,k} w_{1,jk} \right) (1 - y_{0,j})(1 + y_{0,j})\, y_{-1,i}$$

$$= \delta_{0,j}\, y_{-1,i}$$

Finally, adding the direct and indirect contributions we have

$$\frac{\partial E_1}{\partial w_{ji}} = \delta_{1,j} y_{0,i} + \delta_{0,j} y_{-1,i} = \sum_{\tau=1}^{0} \delta_{\tau,j} y_{\tau-1,i}$$

The recursion, and the gradient

The gradient for E_1 can be generalized for any t as

$$\frac{\partial E_t}{\partial w_{ji}} = \sum_{\tau=t}^{0} \delta_{\tau,j} y_{\tau-1,i}$$

with the $\delta_{\tau,j}$ defined recursively as

$$\delta_{\tau,j} \triangleq \begin{cases} (y_{\tau,j} - d_{\tau,j})\ (1 - y_{\tau,j})(1 + y_{\tau,j}), & \text{if } \tau = t, \\ \left(\sum_k w_{jk} \delta_{\tau+1,k} \right) (1 - y_{\tau,j})(1 + y_{\tau,j}), & \text{if } \tau < t. \end{cases}$$

Then, the global gradient is

$$\frac{\partial E}{\partial w_{ji}} = \sum_{t=0}^{T} \sum_{\tau=t}^{0} \delta_{\tau,j} y_{\tau-1,i}$$

—∘ for the input weights W^I the result is the same, with $x_{\tau,i}$ in place of $y_{\tau-1,i}$

BPTT in practice today

Forward python

```
for t in range(len(inputs)):
  xs[t] = inputs[t]
  ys[t] = np.tanh(self.WI @ xs[t] +
                  self.W @ ys[t-1] +
                  self.b)
```

Backward python

```
for t in range(len(xs)):
  yd = targets[t]
  dha = (ys[t] - yd)
  for k in range(t, -1, -1):
    dha = dha * (1 - ys[k]) * (1 + ys[k])
    db  += dha
    dWI += np.dot(dha, xs[k].T)
    dW  += np.dot(dha, ys[k-1].T)
    dha = np.dot(self.W.T, dha)
```

BPTT in practice today

Forward python

```
for t in range(len(inputs)):
  xs[t] = inputs[t]
  ys[t] = np.tanh(self.WI @ xs[t] +
                  self.W @ ys[t-1] +
                  self.b)
```

Forward PyTorch

```
for t in range(len(inputs)):
  xs[t] = inputs[t]
  ys[t] = torch.tanh(self.WI @ xs[t] +
                     self.W @ ys[t-1] +
                     self.b)
```

Backward python

```
for t in range(len(xs)):
  yd = targets[t]
  dha = (ys[t] - yd)
  for k in range(t, -1, -1):
    dha = dha * (1 - ys[k]) * (1 + ys[k])
    db  += dha
    dWI += np.dot(dha, xs[k].T)
    dW  += np.dot(dha, ys[k-1].T)
    dha = np.dot(self.W.T, dha)
```

Backward PyTorch

BPTT in practice today

Forward python

```
for t in range(len(inputs)):
  xs[t] = inputs[t]
  ys[t] = np.tanh(self.WI @ xs[t] +
                  self.W @ ys[t-1] +
                  self.b)
```

Backward python

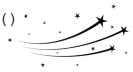
```
for t in range(len(xs)):
  yd = targets[t]
  dha = (ys[t] - yd)
  for k in range(t, -1, -1):
    dha = dha * (1 - ys[k]) * (1 + ys[k])
    db  += dha
    dWI += np.dot(dha, xs[k].T)
    dW  += np.dot(dha, ys[k-1].T)
    dha = np.dot(self.W.T, dha)
```

Forward PyTorch

```
for t in range(len(inputs)):
  xs[t] = inputs[t]
  ys[t] = torch.tanh(self.WI @ xs[t] +
                     self.W @ ys[t-1] +
                     self.b)
```

Backward PyTorch

```
loss.backward()
```



But gradients vanish, and the past is squeezed...

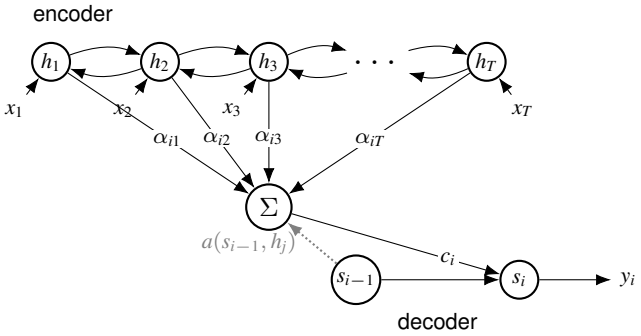
- Each step back multiplies δ by the recurrent weights and by $(1 - y_{\tau,j})(1 + y_{\tau,j})$
  $\Rightarrow$ going back k steps means multiplying k factors
- Factors below one: the gradient vanishes (above one, it explodes)
  $\Rightarrow$ the symmetric sigmoid bounds δ; other activations need gradient clipping or normalization
- An RNN can look arbitrarily far back, but the gradient decides how much it learns
  $\Rightarrow$ representing a dependency is not the same as learning it
  $\Rightarrow$ Gates (LSTM, GRU) open an additive (insted of multiplicative) path through time along it

But gradients vanish, and the past is squeezed...

- Each step back multiplies δ by the recurrent weights and by $(1 - y_{\tau,j})(1 + y_{\tau,j})$
  $\Rightarrow$ going back k steps means multiplying k factors
- Factors below one: the gradient vanishes (above one, it explodes)
  $\Rightarrow$ the symmetric sigmoid bounds δ; other activations need gradient clipping or normalization
- An RNN can look arbitrarily far back, but the gradient decides how much it learns
  $\Rightarrow$ representing a dependency is not the same as learning it
  $\Rightarrow$ Gates (LSTM, GRU) open an additive (insted of multiplicative) path through time along it
- All the past goes through $\mathbf{y}_{t-1}$, a vector of fixed size
  —∘ a long sequence must fit in N numbers
- Attention: let a decoder look at all the encoder outputs and learn to weight them

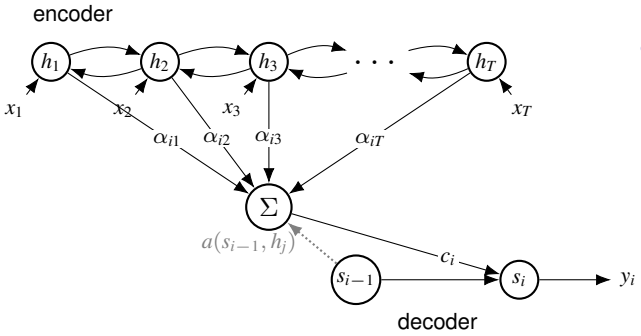
—∘ D. Bahdanau, K. Cho, Y. Bengio. Neural machine translation by jointly learning to align and translate. *(ICLR)*, 2015. arXiv:1409.0473.

Attention: the decoder looks at every state



—∘ D. Bahdanau, K. Cho, Y. Bengio. Neural machine translation by jointly learning to align and translate. *(ICLR)*, 2015. arXiv:1409.0473.

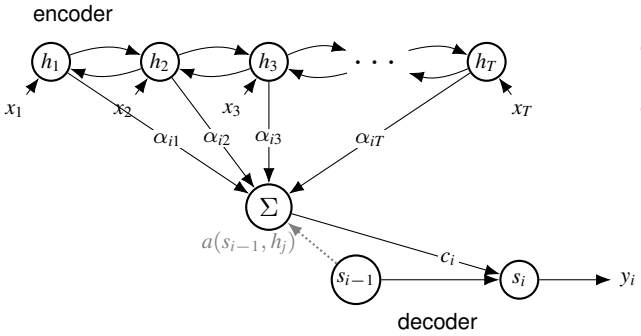
Attention: the decoder looks at every state



- h_j
  - encoder output for input j (bidirectional)

—∘ D. Bahdanau, K. Cho, Y. Bengio. Neural machine translation by jointly learning to align and translate. *(ICLR)*, 2015. arXiv:1409.0473.

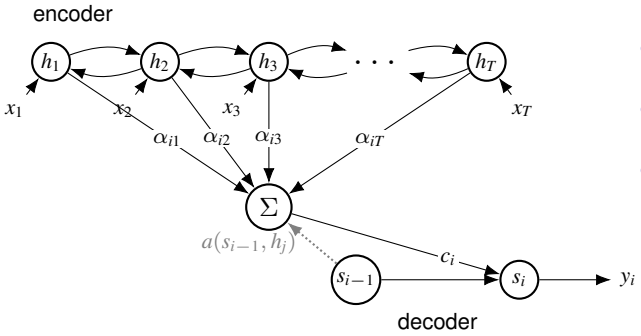
Attention: the decoder looks at every state



- h_j
  - encoder output for input j (bidirectional)
- $e_{ij} = a(s_{i-1}, h_j)$
  - $a(\cdot)$ is a small network: how well h_j fits the next output

D. Bahdanau, K. Cho, Y. Bengio. Neural machine translation by jointly learning to align and translate. *(ICLR)*, 2015. arXiv:1409.0473.

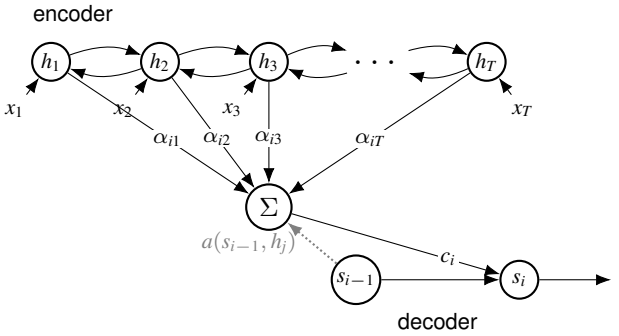
Attention: the decoder looks at every state



- h_j
  - encoder output for input j (bidirectional)
- $e_{ij} = a(s_{i-1}, h_j)$
  - $a(\cdot)$ is a small network: how well h_j fits the next output
- $\alpha_{ij} = \text{softmax}_j(e_{ij})$
  - weights that add up to one

D. Bahdanau, K. Cho, Y. Bengio. Neural machine translation by jointly learning to align and translate. *(ICLR)*, 2015. arXiv:1409.0473.

Attention: the decoder looks at every state

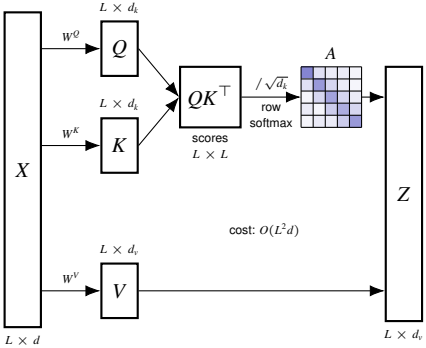


- h_j
  - encoder output for input j (bidirectional)
- $e_{ij} = a(s_{i-1}, h_j)$
  - $a(\cdot)$ is a small network: how well h_j fits the next output
- $\alpha_{ij} = \text{softmax}_j(e_{ij})$
  - weights that add up to one
- $c_i = \sum_j \alpha_{ij} h_j$
  - the context for output i
- $s_i = f(s_{i-1}, y_{i-1}, c_i)$
  - the decoder at step i

- The context is a weighted sum of all the h_j, not the last one
- The weights depend on the content, and are learned with everything else

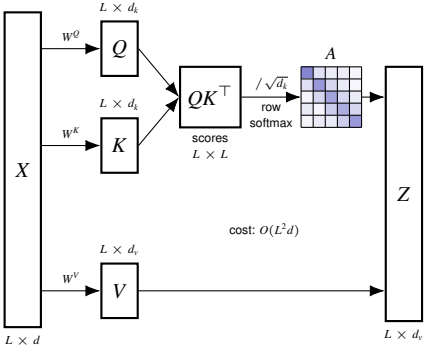
—○ D. Bahdanau, K. Cho, Y. Bengio. Neural machine translation by jointly learning to align and translate. *(ICLR)*, 2015. arXiv:1409.0473.

Self-attention: no recurrence, every position looks at every position



$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})\, V$$

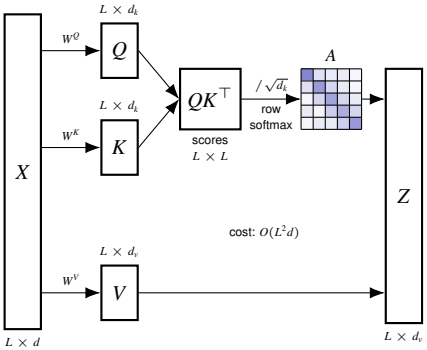
Self-attention: no recurrence, every position looks at every position



- Query, key, value
  - every position asks, every position answers

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})\, V$$

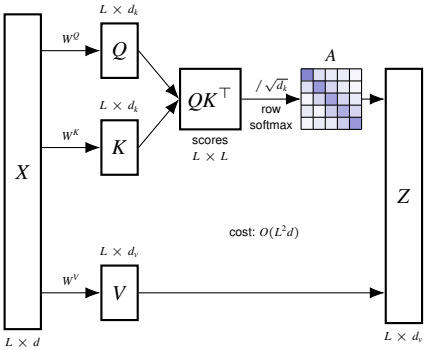
Self-attention: no recurrence, every position looks at every position



- Query, key, value
  - —○ every position asks, every position answers
- The weight is a dot product between positions
  - ⇒ far dependencies are as easy as near ones

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})\, V$$

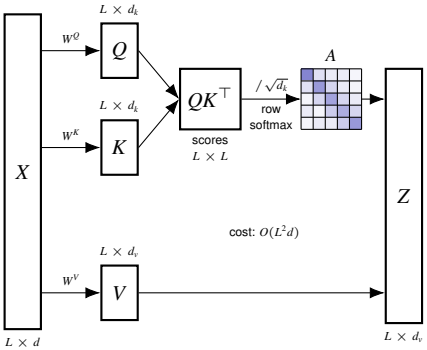
Self-attention: no recurrence, every position looks at every position



- Query, key, value
  - —○ every position asks, every position answers
- The weight is a dot product between positions
  - ⇒ far dependencies are as easy as near ones
- Softmax over the row
  - ⇒ the output is a weighted average of all values

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})\, V$$

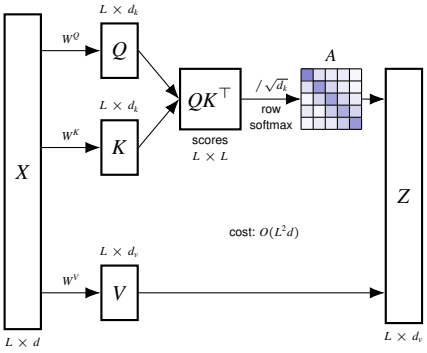
Self-attention: no recurrence, every position looks at every position



- Query, key, value
  - —○ every position asks, every position answers
- The weight is a dot product between positions
  - ⇒ far dependencies are as easy as near ones
- Softmax over the row
  - ⇒ the output is a weighted average of all values
- No recurrence: all positions are computed at once
  - ⇒ but the cost grows as L^2

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})\, V$$

Self-attention: no recurrence, every position looks at every position

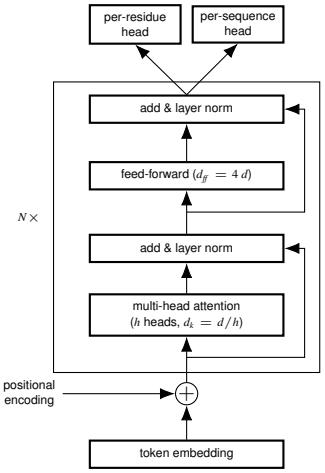


$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})\, V$$

- Query, key, value
  - —∘ every position asks, every position answers
- The weight is a dot product between positions
  - ⇒ far dependencies are as easy as near ones
- Softmax over the row
  - ⇒ the output is a weighted average of all values
- No recurrence: all positions are computed at once
  - ⇒ but the cost grows as L^2
- Order is not in the mechanism
  - ⇒ it has to be added to the input

The transformer: attention + feed-forward stack

The transformer: attention + feed-forward stack





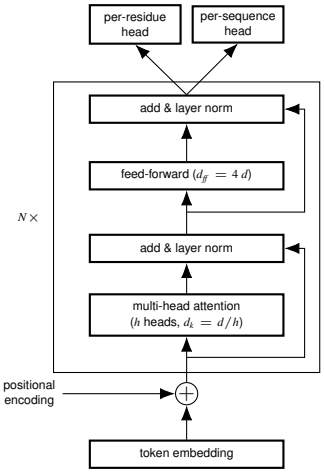
encoder: see everything
(BERT, ESM)



decoder: causal
(GPT, ProtGPT2)

- Multi-head attention (h heads, $d_k = d/h$)
  - each head, a different relation

The transformer: attention + feed-forward stack





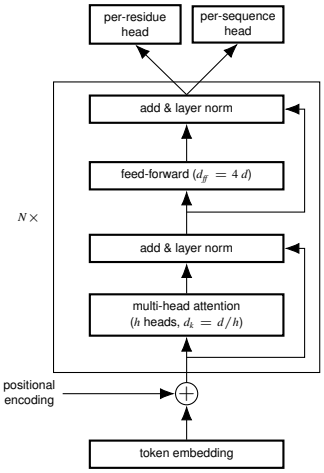
encoder: see everything (BERT, ESM)



decoder: causal (GPT, ProtGPT2)

- Multi-head attention (h heads, $d_k = d/h$)
  - each head, a different relation
- Residual connection and layer norm
  - what makes a deep stack trainable

The transformer: attention + feed-forward stack





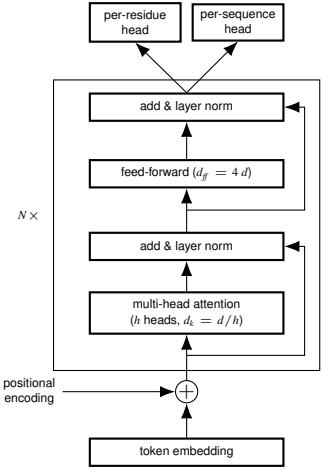
encoder: see everything (BERT, ESM)



decoder: causal (GPT, ProtGPT2)

- Multi-head attention (h heads, $d_k = d/h$)
  - each head, a different relation
- Residual connection and layer norm
  - what makes a deep stack trainable
- An MLP that sees each position separately
  - d inputs, $4d$ hidden neurons, d outputs
  - attention mixes positions, the MLP transforms each one

The transformer: attention + feed-forward stack





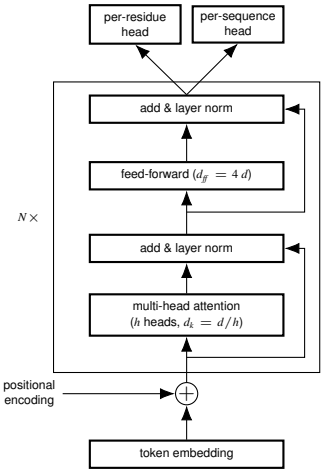
encoder: see everything (BERT, ESM)



decoder: causal (GPT, ProtGPT2)

- Multi-head attention (h heads, $d_k = d/h$)
  - each head, a different relation
- Residual connection and layer norm
  - what makes a deep stack trainable
- An MLP that sees each position separately
  - d inputs, $4d$ hidden neurons, d outputs
  - attention mixes positions, the MLP transforms each one
- Masking decides the model
  - full mask: encoder (BERT, ESM)
  - causal mask: decoder (GPT, ProtGPT2)

The transformer: attention + feed-forward stack





encoder: see everything (BERT, ESM)



decoder: causal (GPT, ProtGPT2)

- Multi-head attention (h heads, $d_k = d/h$)
  - each head, a different relation
- Residual connection and layer norm
  - what makes a deep stack trainable
- An MLP that sees each position separately
  - d inputs, $4d$ hidden neurons, d outputs
  - attention mixes positions, the MLP transforms each one
- Masking decides the model
  - full mask: encoder (BERT, ESM)
  - causal mask: decoder (GPT, ProtGPT2)
- Two heads to transfer
  - one per residue
  - one per sequence

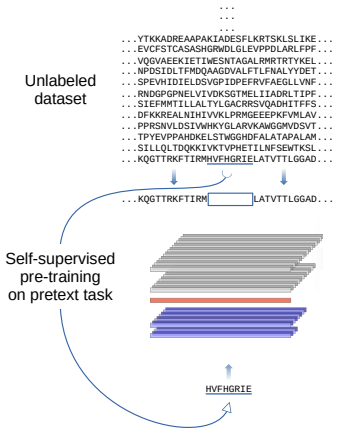
Self-supervised pre-training and transfer learning

Self-supervised pre-training and transfer learning

Unlabeled dataset

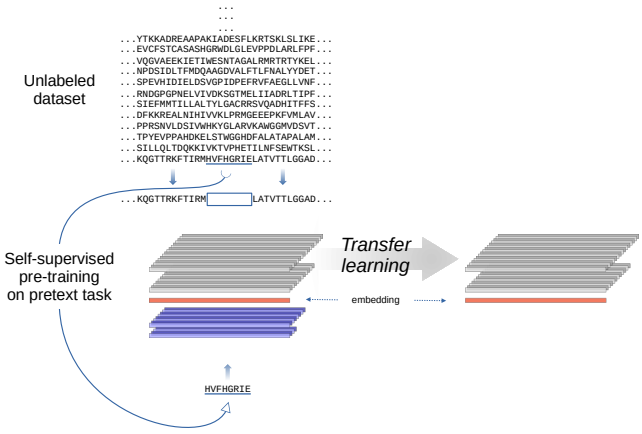


Self-supervised pre-training and transfer learning



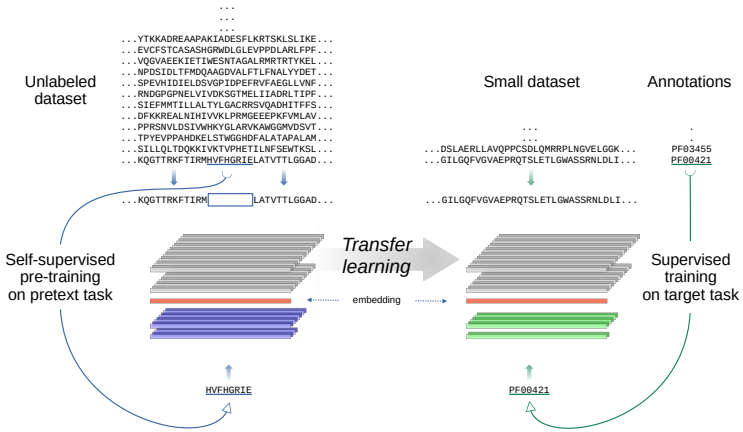
- Pretext task: the masked token

Self-supervised pre-training and transfer learning



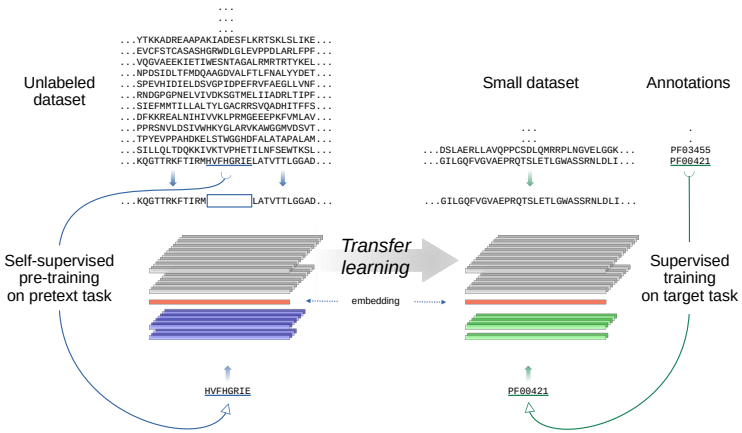
- Pretext task: the masked token
- The embedding is the product

Self-supervised pre-training and transfer learning



- Pretext task: the masked token
- The embedding is the product
- A small labelled set is enough

Self-supervised pre-training and transfer learning



- Pretext task: the masked token
- The embedding is the product
- A small labelled set is enough
- Frozen embeddings or fine-tuning

Foundation models for DNA, RNA and protein

- DNA language models
  - DNABERT-2 (2023), Nucleotide Transformer (2024), Evo 2 (2026), up to genome scale
- RNA language models
  - RNABERT (2022), RNA-FM (2022), RNA-MSM (2023), RNAErnie (2024), ERNIE-RNA (2025), RiNALMo (2025)
- Protein language models
  - ESM-1b (2021), ESM-1v (2021), ProtTrans (2021), ESM-2 and ESMFold (2023), ESM3 (2025)
  - generative: ProtGPT2 (2022), ProGen2 (2023), ProGen3 (2025)

Foundation models for DNA, RNA and protein

- DNA language models
  - DNABERT-2 (2023), Nucleotide Transformer (2024), Evo 2 (2026), up to genome scale
- RNA language models
  - RNABERT (2022), RNA-FM (2022), RNA-MSM (2023), RNAErnie (2024), ERNIE-RNA (2025), RiNALMo (2025)
- Protein language models
  - ESM-1b (2021), ESM-1v (2021), ProtTrans (2021), ESM-2 and ESMFold (2023), ESM3 (2025)
  - generative: ProtGPT2 (2022), ProGen2 (2023), ProGen3 (2025)

What changes in practice?
Reusable embeddings,
few labels,
or even none (zero-shot)

Machine Learning for Biological Sequences

HMM, RNN, LLM: A long sequence of models...

Gracias!

Diego Milone

3ra Escu3la CICADA

tokenpal9za

sinc(*i*)

Instituto de investigación en señales e inteligencia computacional

FICH, Universidad Nacional del Litoral
Consejo Nacional de Investigaciones Científicas y Técnicas, Argentina